arXiv:2304.08467v3 [cs.CL] 12 Feb 2024

Learning to Compress Prompts with Gist Tokens

Jesse Mu, Xiang Lisa Li, Noah Goodman
Stanford University
muj@cs.stanford.edu, {xlisali,ngoodman}@stanford.edu

Abstract

Prompting is the primary way to utilize the multitask capabilities of language models (LMs), but prompts occupy valuable space in the input context window, and repeatedly encoding the same prompt is computationally inefficient. Finetuning and distillation methods allow for specialization of LMs without prompting, but require retraining the model for each task. To avoid this trade-off entirely, we present *gisting*, which trains an LM to compress prompts into smaller sets of "gist" tokens which can be cached and reused for compute efficiency. Gist models can be trained with no additional cost over standard instruction finetuning by simply modifying Transformer attention masks to encourage prompt compression. On decoder (LLaMA-7B) and encoder-decoder (FLAN-T5-XXL) LMs, gisting enables up to 26x compression of prompts, resulting in up to 40% FLOPs reductions, 4.2% wall time speedups, and storage savings, all with minimal loss in output quality.

1 Introduction

Consider the prompt of a Transformer [34] language model (LM) like ChatGPT:[1]

```
You are ChatGPT, a large language model trained by OpenAI. You answer as concisely as
possible for each response (e.g. don't be verbose). It is very important that you answer
as concisely as possible, so please remember this. If you are generating a list, do not
have too many items. Keep the number of items short.
Knowledge cutoff: 2021-09 Current date: <TODAY>
```

With millions of queries a day, an unoptimized ChatGPT would encode this prompt over and over with a self-attention mechanism whose time and memory complexity is quadratic in the length of the input. Caching the Transformer activations of the prompt can prevent some recomputation, yet this strategy still incurs memory and storage costs as the number of cached prompts grows. At large scales, even small reductions in prompt length could lead to substantial compute, memory, and storage savings over time, while also letting users fit more content into an LM's limited context window.

How might we reduce the cost of this prompt? One typical approach is to finetune or distill [1, 30] the model to behave similarly to the original model without the prompt, perhaps with parameter-efficient adaptation methods [15, 16, 19]. Yet a fundamental drawback of this approach is that it requires retraining the model for each new prompt (Figure 1, bottom left).

Instead, we propose **gisting** (Figure 1, top right), which compresses arbitrary prompts into a smaller set of Transformer activations on top of virtual "gist" tokens, *a la* prefix-tuning [19]. But where prefix-tuning requires learning prefixes via gradient descent for each task, gisting adopts a meta-learning approach, where we simply predict the gist prefixes zero-shot given only the prompt, allowing for generalization to unseen instructions without any additional training. Since gist tokens are much shorter than the full prompt, gisting allows arbitrary prompts to be compressed, cached, and reused for compute efficiency.

[1] reddit.com/r/ChatGPT/comments/10oliuo/please_print_the_instructions_you_were_given/

37th Conference on Neural Information Processing Systems (NeurIPS 2023).

arXiv:2304.08467v3 [cs.CL] 12 Feb 2024

学习如何使用摘要标记压缩提示词

杰西·穆、丽莎·李、诺亚·古德曼斯坦福大学
muj@cs.stanford.edu, {xlisali,ngoodman}@stanford.edu

摘要

Prompting is the primary way to utilize the multitask capabilities of language models (LMs), but prompts occupy valuable space in the input context window, and repeatedly encoding the same prompt is computationally inefficient. Finetuning and distillation methods allow for specialization of LMs without prompting, but require retraining the model for each task. To avoid this trade-off entirely, we present *gisting*, which trains an LM to compress prompts into smaller sets of "gist" tokens which can be cached and reused for compute efficiency. Gist models can be trained with no additional cost over standard instruction finetuning by simply modifying Transformer attention masks to encourage prompt compression. On decoder (LLaMA-7B) and encoder-decoder (FLAN-T5-XXL) LMs, gisting enables up to 26x compression of prompts, resulting in up to 40% FLOPs reductions, 4.2% wall time speedups, and storage savings, all with minimal loss in output quality.

1 引言

以类似ChatGPT的Transformer [34]语言模型（LM）的提示词为例：[1]

```
你是ChatGPT，一个由OpenAI训练的大型语言模型。你需要尽可能简洁地回复每一条问题（例如
避免冗长表述）。尽可能简明扼要地作答至关重要，请务必牢记这一点。如果你需要列出内容，项
数不要过多，尽量保持列表简洁。知识截止日期：2021年9月，当前日期： <TODAY>
```

由于每天会有数百万次查询，若不进行优化，ChatGPT会反复使用自注意力机制来编码这些提示词，而该机制的时间与内存复杂度会随输入长度呈二次方增长。虽然对提示词的Transformer激活值进行缓存可以避免部分重复计算，但随着缓存提示词数量的增加，这一策略依然会产生内存和存储成本。在大规模应用场景中，即便只是略微缩短提示词长度，长期来看也能大幅节省计算、内存和存储资源，同时帮助用户在语言模型有限的上下文窗口中容纳更多内容。

我们该如何降低这个提示词的生成成本呢？一种常见的方法是对 [1, 30] 模型进行微调或蒸馏，使其在不依赖原提示词的情况下仍能表现出与原模型相似的行为，或许还可以借助参数高效的自适应方法来实现这一点 [15, 16, 19]。不过，这种方法存在一个根本性的缺陷，那就是每次遇到新的提示词时都需要重新训练模型（见图1左下角）。

Instead, we propose **gisting** (Figure 1, top right), which compresses arbitrary prompts into a smaller set of Transformer activations on top of virtual "gist" tokens, *a la* prefix-tuning [19]. But where prefix-tuning requires learning prefixes via gradient descent for each task, gisting adopts a meta-learning approach, where we simply predict the gist prefixes zero-shot given only the prompt, allowing for generalization to unseen instructions without any additional training. Since gist tokens are much shorter than the full prompt, gisting allows arbitrary prompts to be compressed, cached, and reused for compute efficiency.

[1] reddit.com/r/ChatGPT/comments/10oliuo/请_打印出_你之前_收到的_使用说明_/

第37届神经信息处理系统会议（NeurIPS 2023）。

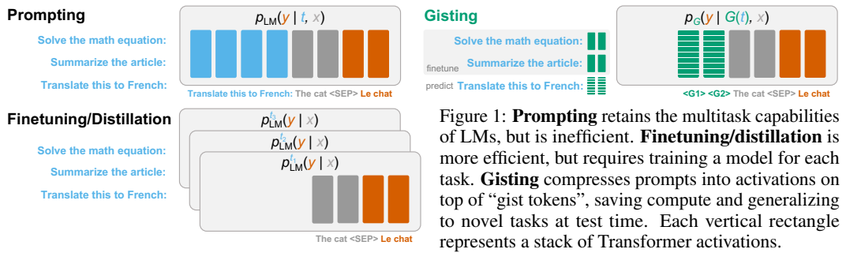


Figure 1: **Prompting** retains the multitask capabilities of LMs, but is inefficient. **Finetuning/distillation** is more efficient, but requires training a model for each task. **Gisting** compresses prompts into activations on top of "gist tokens", saving compute and generalizing to novel tasks at test time. Each vertical rectangle represents a stack of Transformer activations.

In this paper, we further propose a very simple way to learn a gist model: doing instruction tuning [38] with gist tokens inserted after the prompt, and a modified attention mask preventing tokens *after* the gist tokens from attending to tokens *before* the gist tokens. This allows a model to learn prompt compression and instruction following at the same time, with no additional training cost.

On decoder-only (LLaMA-7B) and encoder-decoder (FLAN-T5-XXL) LMs, gisting achieves prompt compression rates of up to **26x**, while maintaining output quality similar to the original models in human evaluations. This results in up to 40% FLOPs reduction and 4.2% latency speedups during inference, with greatly decreased storage costs compared to traditional prompt caching approaches.

2 Gisting

We will first describe gisting in the context of instruction finetuning [38]. We have an instruction-following dataset $\mathcal{D} = \{(t_i, x_i, y_i)\}_{i=1}^{N}$, where t is a task encoded with a natural language prompt (e.g. `Translate this to French`), x is an (optional) input for the task (e.g. `The cat`), and y is the desired output (e.g. `Le chat`). Given a (usually pretrained) LM, the aim of instruction finetuning is to learn a distribution $p_{\text{LM}}(y \mid t, x)$, typically by concatenating t and x, then having the LM autoregressively predict y. At inference time, we can *prompt* the model with a novel task t and input x, decoding from the model to obtain its prediction.

However, this pattern of concatenating t and x has drawbacks: Transformer LMs have limited context windows, bounded either by architecture or memory limits. Furthermore, given that attention scales quadratically in the length of the input, long prompts t, especially those that are repeatedly reused, are computationally inefficient. What options do we have to reduce the cost of prompting?

One simple option is to finetune the LM for a *specific* task t. That is, given $\mathcal{D}^t = \{(x_i, y_i)\}_{i=1}^{N^t}$, the dataset containing input/output examples only under task t, we can learn a specialized LM $p_{\text{LM}}^t(y \mid x)$ which is faster because it does not condition on t. Parameter-efficient finetuning methods such as prefix-/prompt-tuning [18, 19] or adapters [15, 16] promise to do so at a fraction of the cost of full finetuning, and newer methods like HyperTuning [25] eliminate gradient descent entirely, instead predicting the parameters of the specialized model directly from $\mathcal{D}^t$. Yet problems with these methods still remain: we must store at least a subset of model weights for each task, and more importantly, for each task t, we must collect a corresponding dataset of input/output pairs $\mathcal{D}^t$ to adapt the model.

Gisting is a different approach that amortizes both (1) the inference-time cost of prompting p_{LM} with t and (2) the train-time cost of learning a new p_{LM}^t for each t. The idea is to learn a *compressed* version of t, $G(t)$, such that inference from $p_{\boldsymbol{G}}(y \mid G(t), x)$ is faster than $p_{\text{LM}}(y \mid t, x)$. In LM terms, $G(t)$ will be the key/value activations on top a set of *gist tokens*, smaller than the number of tokens in t, yet still inducing similar behavior from the LM. Also known as a Transformer *prefix* [19], $G(t)$ can then be cached and reused for compute efficiency. Crucially, we expect G to generalize to unseen tasks: given a new task t, we can predict and use the gist activations $G(t)$ *without any additional training*.

2.1 A Context Distillation Perspective

An alternative way to view gisting is through the lens of distillation of an already instruction-tuned LM $p_{\text{LM}}(y \mid t, x)$. Askell et al. [1] and Snell et al. [30] define *context distillation* as the process of finetuning a new LM p_{CD}^t to mimic the original LM without the prompt ("context") t, via the loss

$$\mathcal{L}_{\text{CD}}(p_{\text{CD}}^t,\, t) = \mathbb{E}_x \left[D_{\text{KL}}(p_{\text{LM}}(y \mid t, x) \parallel p_{\text{CD}}^t(y \mid x)) \right]. \tag{1}$$



图1：**提示词输入**方式能够保留语言模型的多任务处理能力，但效率较低。**微调/蒸馏**方法效率更高，但需要为每项任务单独训练一个模型。**摘要化技术**则能在"摘要标记"的基础上将提示词压缩为激活值，从而节省计算资源，并在测试时适配新的任务类型。每个垂直矩形代表一组Transformer激活值。

在本文中，我们进一步提出了一种极其简单的方法来训练一个擅长要点摘要的模型：通过在提示词后插入摘要标记并对其进行指令微调[38]，同时使用改良后的注意力掩码，阻止位于摘要标记之后的标记去关注位于摘要标记之前的标记。这样一来，模型便无需额外的训练成本，即可同时掌握提示词压缩与指令遵循的能力。

在仅解码器结构（LLaMA-7B）及编码器-解码器结构（FLAN-T5-XXL）的语言模型上，摘要化技术可实现高达<span style id='1'>26倍的提示词压缩率，同时在人类评估中的输出质量与原始模型相当。这样一来，在推理过程中不仅能将浮点运算次数减少多达40%，延迟时间也能加快4.2%，相较于传统的提示词缓存方法，其存储成本更是大幅降低。

2 摘要化技术

We will first describe gisting in the context of instruction finetuning [38]. We have an instruction-following dataset $\mathcal{D} = \{(t_i, x_i, y_i)\}_{i=1}^{N}$, where t is a task encoded with a natural language prompt (e.g. `Translate this to French`), x is an (optional) input for the task (e.g. `The cat`), and y is the desired output (e.g. `Le chat`). Given a (usually pretrained) LM, the aim of instruction finetuning is to learn a distribution $p_{\text{LM}}(y \mid t, x)$, typically by concatenating t and x, then having the LM autoregressively predict y. At inference time, we can *prompt* the model with a novel task t and input x, decoding from the model to obtain its prediction.

然而，将 t 与 x 直接拼接的这种方式存在诸多弊端：Transformer语言模型的上下文窗口规模有限，这一限制既源于其架构设计，也受内存容量所限。此外，由于注意力机制的计算复杂度会随着输入长度的增加而呈二次方增长，过长的提示词 t——尤其是那些被反复重复使用的提示词——会带来极高的计算成本。那么，我们有哪些方法可以降低生成提示的代价呢?

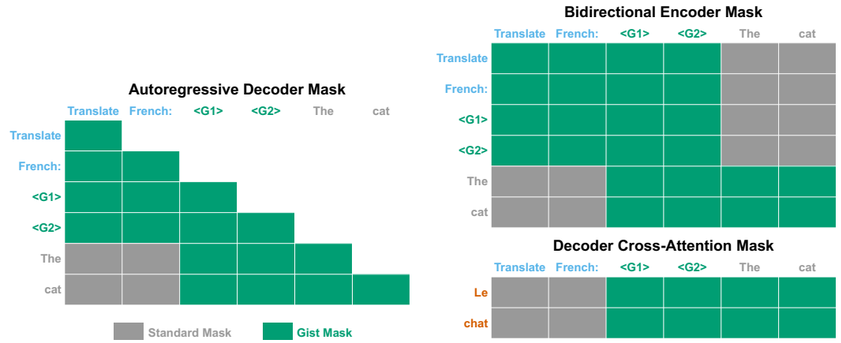
One simple option is to finetune the LM for a *specific* task t. That is, given $\mathcal{D}^t = \{(x_i, y_i)\}_{i=1}^{N^t}$, the dataset containing input/output examples only under task t, we can learn a specialized LM $p_{\text{LM}}^t(y \mid x)$ which is faster because it does not condition on t. Parameter-efficient finetuning methods such as prefix-/prompt-tuning [18, 19] or adapters [15, 16] promise to do so at a fraction of the cost of full finetuning, and newer methods like HyperTuning [25] eliminate gradient descent entirely, instead predicting the parameters of the specialized model directly from $\mathcal{D}^t$. Yet problems with these methods still remain: we must store at least a subset of model weights for each task, and more importantly, for each task t, we must collect a corresponding dataset of input/output pairs $\mathcal{D}^t$ to adapt the model.

Gisting is a different approach that amortizes both (1) the inference-time cost of prompting p_{LM} with t and (2) the train-time cost of learning a new p_{LM}^t for each t. The idea is to learn a *compressed* version of t, $G(t)$, such that inference from $p_{\boldsymbol{G}}(y \mid G(t)\ x)$ is faster than $p_{\text{LM}}(y \mid t, x)$. In LM terms, $G(t)$ will be the key/value activations on top a set of *gist tokens*, smaller than the number of tokens in t, yet still inducing similar behavior from the LM. Also known as a Transformer *prefix* [19], $G(t)$ can then be cached and reused for compute efficiency. Crucially, we expect G to generalize to unseen tasks: given a new task t, we can predict and use the gist activations $G(t)$ *without any additional training*.

2.1 从上下文蒸馏的角度来看

另一种理解摘要化技术的方式，是将其视为对已被指令微调过的语言模型 $p_{\text{LM}}(y \mid t,\ x)$进行上下文蒸馏的体现。Askell等人 [1] 与Snell等人 [30] 将上下文蒸馏定义为通过损失函数，对新的语言模型p^t CD 进行微调，使其能够在无需提示词（即"上下文"） t的情况下模仿原始语言模型的过程。

$$\mathcal{L}_{\text{CD}}(p_{\text{CD}}^t,\, t) = \mathbb{E}_x \left[D_{\text{KL}}(p_{\text{LM}}(y \mid t, x) \parallel p_{\text{CD}}^t(y \mid x)) \right]. \tag{1}$$



(a) Decoder-only (e.g. GPT, LLaMA) (b) Encoder-decoder (e.g. T5)

Figure 2: **Gist Masking**. Attention mask modifications for (a) decoder-only and (b) encoder-decoder Transformer LMs to encourage prompt compression into gist tokens `<G1> <G2>`. In these tables, cell (r, c) shows whether token r can attend to token c during self- or cross-attention.

The insight to be gained from this perspective is that we do not need any external data $\mathcal{D}$: this KL objective can be approximated by finetuning p_{CD}^t on a synthetic sampled dataset $\hat{\mathcal{D}}^t = \{(\hat{x}_i, \hat{y}_i)\}$ where $(\hat{x}_i, \hat{y}_i) \sim p_{\text{LM}}(\cdot \mid t)$. This is precisely the approach taken by recent work [1, 7, 30], including Wingate et al. [40], who notably learn to compress a single discrete prompt into a soft prompt via gradient descent, similar to this paper.

However, we differ from this prior work in that we are not interested in distilling just a single task, but in amortizing the cost of distillation across a *distribution* of tasks T. That is, given a task $t \sim T$, instead of obtaining the distilled model via gradient descent, we use G to simply *predict* the gist tokens ($\approx$ parameters) of the distilled model, in the style of HyperNetworks [13] and HyperTuning [25]. Our "meta" distillation objective is thus (with changes highlighted in **blue**):

$$\mathcal{L}_{\color{blue}G}(p_{\color{blue}G}, \color{blue}T) = \mathbb{E}_{\color{blue}t \sim T, x}\left[D_{\text{KL}}(p_{\text{LM}}(y \mid t, x) \parallel p_{\color{blue}G}(y \mid \color{blue}G(t), x))\right]. \quad (2)$$

In the experiments we describe below, we train on synthetic instruction-following data sampled from instruction-tuned variants of GPT-3 [3, 23]. Thus, these experiments can indeed be seen as a form of context distillation for the GPT-3 series models.

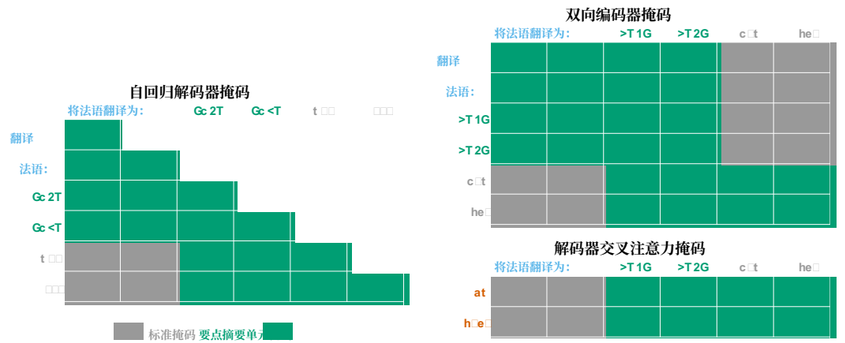
3 Learning Gisting by Masking

Having just described the general framework of gisting, here we will explore an extremely simple way of learning such a model: using the LM itself as the gist predictor G. This not only leverages the pre-existing knowledge in the LM, but also allows us to learn gisting by simply doing standard instruction finetuning while modifying the Transformer attention masks to enforce prompt compression. This means that gisting incurs *no* additional training cost on top of standard instruction finetuning!

Specifically, we add a *single* gist token g to the model vocabulary and embedding matrix. Then, given a (task, input) pair (t, x), we concatenate t and x with a set of k copies of g in between: $(t, g_1, \ldots, g_k, x)$, e.g. `Translate French: <G1> <G2> The cat`.[2] The model is then restricted such that input tokens *after* the gist tokens cannot attend to any of the prompt tokens *before* the gist tokens (but they *can* attend to the gist tokens). This forces the model to compress the prompt information into the gist prefix, since the input x (and output y) cannot attend to the prompt t.

Figure 2 illustrates the required changes. For **decoder-only** LMs such as GPT-3 [3] or LLaMA [33] that normally admit an autoregressive, causal attention mask, we simply mask out the lower-left corner of the triangle (Figure 2a). For **encoder-decoder** LMs (e.g. T5; [28]) with a bidirectional encoder followed by an autoregressive decoder, two changes are needed (Figure 2b). First, in the encoder, which normally has no masking, we prevent the input x from attending to the prompt t. But

[2] Again, the gist token is the same from g_1 to g_k; what changes is the activations on top of each token.



(a) 仅解码器架构（例如GPT、LLaMA模型） (b) 编码器-解码器结构（例如T5模型）

Figure 2: **Gist Masking**. Attention mask modifications for (a) decoder-only and (b) encoder-decoder Transformer LMs to encourage prompt compression into gist tokens `<G1> <G2>`. In these tables, cell (r, c) shows whether token r can attend to token c during self- or cross-attention.

The insight to be gained from this perspective is that we do not need any external data $\mathcal{D}$: this KL objective can be approximated by finetuning p_{CD}^t on a synthetic sampled dataset $\hat{\mathcal{D}}^t = \{(\hat{x}_i, \hat{y}_i)\}$ where $(\hat{x}_i, \hat{y}_i) \sim p_{\text{LM}}(\cdot \mid t)$. This is precisely the approach taken by recent work [1, 7, 30], including Wingate et al. [40], who notably learn to compress a single discrete prompt into a soft prompt via gradient descent, similar to this paper.

However, we differ from this prior work in that we are not interested in distilling just a single task, but in amortizing the cost of distillation across a *distribution* of tasks T. That is, given a task $t \sim T$, instead of obtaining the distilled model via gradient descent, we use G to simply *predict* the gist tokens ($\approx$ parameters) of the distilled model, in the style of HyperNetworks [13] and HyperTuning [25]. Our "meta" distillation objective is thus (with changes highlighted in **blue**):

$$\mathcal{L}_{\color{blue}G}(p_{\color{blue}G}, \color{blue}T) = \mathbb{E}_{\color{blue}t \sim T, x}\left[D_{\text{KL}}(p_{\text{LM}}(y \mid t, x) \parallel p_{\color{blue}G}(y \mid \color{blue}G(t), x))\right]. \quad (2)$$

在下面描述的实验中，我们使用从经过指令微调的GPT-3 [3, 23]版本中抽取的合成指令遵循数据来进行训练。因此，这些实验实际上可以被视为针对GPT-3系列模型的一种上下文蒸馏方式。

3 通过掩码实现摘要化技术学习

在刚刚介绍了摘要化技术的一般框架之后，接下来我们将探讨一种学习此类模型的极其简单的方法：即利用语言模型本身作为要点摘要的预测器 G。这种方法不仅能借助语言模型中已有的知识，还能让我们只需通过标准的指令微调，并配合修改Transformer注意力掩码以实现提示词压缩，就能学会摘要化技术。这意味着，相较于标准的指令微调，摘要化技术无需额外的训练成本！

Specifically, we add a *single* gist token g to the model vocabulary and embedding matrix. Then, given a (task, input) pair (t, x), we concatenate t and x with a set of k copies of g in between: $(t, g_1, \ldots, g_k, x)$, e.g. `Translate French: <G1> <G2> The cat`.[2] The model is then restricted such that input tokens *after* the gist tokens cannot attend to any of the prompt tokens *before* the gist tokens (but they *can* attend to the gist tokens). This forces the model to compress the prompt information into the gist prefix, since the input x (and output y) cannot attend to the prompt t.

图2展示了所需进行的修改。对于诸如GPT-3 [3] 或LLaMA [33]这类通常采用自回归式因果注意力掩码的**仅解码器型**语言模型，我们只需遮盖三角形左下角区域（见图2a）。而对于那些具有双向编码器后再接自回归解码器的**编解码器型**语言模型（例如T5； [28]），则需要进行两项修改（见图2b）。首先，在通常不设置掩码的编码器中，我们需要阻止输入 x 对提示词 t产生注意力关联。但是

[2] 需要再次说明的是，要点摘要标记实际上就是从 g_1 到 g_k的那一部分并无变化；真正发生变化的是各个标记对应的激活 即每个标记上对应的激活值。

we must also prevent the prompt t and gist tokens g_i from attending to the input x, since otherwise the encoder learns different representations depending on the input. Finally, the decoder operates as normal, except during cross-attention, we prevent the decoder from attending to the prompt t.

Overall, these masking changes are extremely simple and can be implemented in roughly 10 source lines of code. See Appendix A for a sample PyTorch implementation which can be used as a drop-in replacement for attention masking in deep learning libraries such as Hugging Face Transformers [41].

4 Experiments

4.1 Data

A dataset with a large variety of tasks (prompts) is crucial to learn gist models that can generalize. To obtain the largest possible set of tasks for instruction finetuning, we create a dataset called Alpaca+, which combines the Self-Instruct [36] and Stanford Alpaca [31] instruction tuning datasets, each consisting of (t, x, y) tuples sampled from OpenAI's `text-davinci-001` and `text-davinci-003` variants of GPT-3, respectively. In total, Alpaca+ has 130,321 examples, with 104,664 unique tasks t, 48,530 unique inputs x, and anywhere from 0–5 inputs per task (0.64 on average).

Note that ~59% of tasks in Alpaca+ have no inputs (e.g. `Write me a poem about frogs`), in which case we simply omit the input x. While it is less interesting to cache such prompts since they are not input-dependent, they still serve as valuable training signal for learning prompt compression. Overall, while Alpaca+ is noisy and imperfect, Wang et al. [36] and Taori et al. [31] nevertheless show that models trained on such data achieve comparable performance to the original models from which the data is sampled, making this a promising testbed for studying gisting.

From Alpaca+ we hold out 3 validation splits: 1000 **Seen** prompts (with unseen, non-empty inputs); 1000 **Unseen** prompts (with non-empty inputs); and the 252 hand-written **Human** prompts and completions used in Wang et al. [36], of which 83% have non-empty inputs. The latter two splits test generalization to unseen instructions, with the **Human** split posing a stronger out-of-distribution (OOD) challenge: the average training prompt has ~20 tokens, compared to ~26 in the human split.

4.2 Models

To demonstrate gisting across multiple Transformer LM architectures, we experiment with LLaMA-7B [33], a decoder-only GPT-style model with ~7B parameters, and FLAN-T5-XXL [8], an encoder-decoder T5 model [28] with 11B parameters. For each of these models, we train models with a varying number of gist tokens $k \in \{1, 2, 5, 10\}$, using the modified attention masks described in Section 3. To assess how the model is learning prompt compression, we calibrate performance against upper- and lower-bound baselines and a simple discrete compression strategy:

Positive Control. As an upper bound on performance, we train a model with a single gist token, but without any modifications to the attention mask. This is akin to doing standard instruction finetuning.

Negative Control. As a lower bound on performance, we train a model without access to the task t. This is similar to a "random gist token" baseline, which allows us to measure how the model would do if it failed to compress *any* information into the gist prefix.

Discrete Compression with TF-IDF. An alternative approach to compression is simply using fewer discrete tokens to express the same task. Achieving compression rates similar to gisting, however, requires compression far beyond any threshold of fluency. Nevertheless, as a baseline, we compute TF-IDF statistics over the set of instructions in the Alpaca+ training set to extract the most relevant keyword in each instruction. Some examples from the training set include (see Appendix G for more):

`Write a letter to your boss asking for an increase in salary` → **`salary`**
`Given two integers, find their average` → **`average`**

We then replace each instruction in Alpaca+ with the first subword token from each keyword, resulting in compression rates equivalent to a model trained with a single gist token. Similarly to the positive control, we do standard instruction finetuning over Alpaca+ with these compressed instructions.

For full training, data, and compute details, and a link to code, see Appendix B.

我们还必须防止提示词 t 与摘要标记 g_i 关注输入 x，否则编码器会根据不同的输入学习出不同的表示形式。最后，解码器的运行方式与正常情况相同，只是在跨注意力机制期间，我们会阻止解码器关注提示词 t。

总体而言，这些掩码修改方式极为简单，大约仅需10行源代码即可实现。有关示例PyTorch实现代码，请参见附录A，该代码可直接用于替代HuggingFace Transformers [41]等深度学习库中的attentionmasking功能。

4 实验

4.1 数据

A dataset with a large variety of tasks (prompts) is crucial to learn gist models that can generalize. To obtain the largest possible set of tasks for instruction finetuning, we create a dataset called Alpaca+, which combines the Self-Instruct [36] and Stanford Alpaca [31] instruction tuning datasets, each consisting of (t, x, y) tuples sampled from OpenAI's `text-davinci-001` and `text-davinci-003` variants of GPT-3, respectively. In total, Alpaca+ has 130,321 examples, with 104,664 unique tasks t, 48,530 unique inputs x, and anywhere from 0–5 inputs per task (0.64 on average).

需要注意的是，Alpaca+ 中 ~59%的任务并没有输入标记张量（例如“为我写一首关于青蛙的诗”），在这种情况下我们直接省略输入 x。虽然这类提示词并不依赖输入，因此缓存起来趣味性较低，但它们依然是用于学习提示词压缩技术的宝贵训练信号。总体而言，尽管Alpaca+存在噪声且不够完美，但Wang等人 [36] 以及Taori等人 [31] 的研究仍表明，基于此类数据训练出的模型能够达到与原始采样模型相当的性能，这为研究摘要化技术提供了一个极具前景的测试平台。

From Alpaca+ we hold out 3 validation splits: 1000 **Seen** prompts (with unseen, non-empty inputs); 1000 **Unseen** prompts (with non-empty inputs); and the 252 hand-written **Human** prompts and completions used in Wang et al. [36], of which 83% have non-empty inputs. The latter two splits test generalization to unseen instructions, with the **Human** split posing a stronger out-of-distribution (OOD) challenge: the average training prompt has ~20 tokens, compared to ~26 in the human split.

4.2 模型

为展示这种摘要化技术在不同Transformer语言模型架构上的应用效果，我们分别对拥有 ~7B参数的LLaMA- 7B [33], a仅解码器的GPT风格模型，以及拥有11B参数的FLAN-T5-XXL [8], 编码器-解码器结构的T5模型进行了实验。针对这些模型中的每一款，我们都使用了第3节中介绍的改进版注意力掩码，训练了含有不同数量摘要标记 $k \in \{1, 2, 5, 10\}$的版本。为评估模型在提示词压缩方面的学习效果，我们将其性能与上下界基准值以及一种简单的离散压缩策略进行了对比。

正面评价**我掌控全局**。作为性能的上限，我们使用一个提提示词来训练模型，b
~~无疑~~~~会对~~注意力掩码进行任何修改。这类似于执行标准的指令微调操作。 ng.

NegativeControl. 作为性能的下限参考，我们会训练一个无法获取任务 t相关信息的模型。这类似于“随机要点摘要提示词”基准测试，通过它我们可以衡量该模型在无法将任何信息压缩进要点摘要前缀时的表现。

基于TF-IDF的离散压缩。另一种压缩方法则是直接使用更少的离散标记来完成同一任务。不过，若要达到与摘要化技术相似的压缩率，所需的压缩程度远远超出了正常流畅度的范畴。尽管如此，作为基准测试，我们仍会对Alpaca+ 训练集中的所有指令计算TF-IDF统计值，从而提取出每条指令中最相关的关键词。训练集中的一些示例如下（更多内容见附录G）：

给你的上司写封信，请求提高 → **薪资**。给定两个整数，求它们的平均值 → **平均值**。

接着，我们用每个关键词的第一个子词标记替换Alpaca+中的每条指令，由此得到的压缩率等同于使用单个要点摘要标记训练的模型所达到的压缩率。与阳性对照组的方法类似，我们也会使用这些经过压缩的指令对Alpaca+ 进行标准的指令微调。
如需了解完整的训练流程、数据与计算资源相关细节，以及代码链接，请参阅附录B。

4.3 Evaluation

Our evaluation uses a combination of automated metrics and AI- and human-assisted evaluation:

ROUGE-L. We first use ROUGE-L, a simple lexical overlap statistic [20], used in previous open-ended instruction finetuning work [37, 38]. The `text-davinci-{001,003}` completions are used as references, except for the Human split, where we use the gold-standard human reference.

ChatGPT. Next, we use ChatGPT-3.5 [22] to compare the outputs of our models to the positive control. While this is an imperfect metric, it allows for much faster and cheaper evaluation than human experiments, with an arguably more meaningful semantic signal than ROUGE-L. Recent work has found that ChatGPT can be used for text annotation and evaluation [12, 17, 35] with near-human performance, and similar model-based evaluations have been conducted with recent LMs [5, 11, 31].

Specifically, given a task t, input x, and outputs from two models (y_1, y_2) identified only as Assistants A and B, ChatGPT was asked to choose which assistant response is better, explaining its reasoning in Chain-of-Thought fashion [39]. If the models produced the same output, or were equally bad, ChatGPT was allowed to call a tie. We gave examples of desired outputs in ChatGPT's prompt, and randomized the order of presentation between the models for each query to avoid order effects. The full prompt given to ChatGPT and evaluation details are in Appendix C. Using these outputs, we measure the win rate of a model against the positive control: a win rate of 50% indicates that the model is of comparable quality to a model that does no prompt compression.

Human eval. Finally, after prototyping with ChatGPT, we select the best gist compression models and do a Human evaluation on a random subset of 100 of the 252 examples in the Human validation split. For each of the 100 examples, we recruited 3 US or UK-based, English-fluent annotators from Prolific, and asked them to rate model outputs in the same style as the ChatGPT evaluation above (see Appendix D for full details, including the annotation interface). The only difference is that human participants were allowed to select "I Don't Know" in cases where they had inadequate domain knowledge to accurately judge the responses, e.g. if the question was a coding question; we drop these responses (~10%) during analysis. With this human evaluation, we are not only interested in evaluating our final models, but also validating whether ChatGPT can be used as a reliable replacement for human annotation on this task.

5 Results

ROUGE-L and ChatGPT evaluations for LLaMA-7B and FLAN-T5-XXL, with varying numbers of gist tokens, are shown in Figure 3. Models were generally insensitive to the number of gist tokens k: compressing prompts into a single token prefix did not substantially underperform larger prefixes. In fact, having too many gist tokens hurts performance in some cases (e.g. LLaMA-7B, 10 gist tokens), perhaps because the increased capacity enables overfitting to the training distribution. Thus, we use the single gist token models for the rest of the experiments in the paper, and report the exact numbers for the single-token models, with the positive, negative, and TF-IDF baselines, in Table 1.

On **Seen** instructions, gist models attain near-identical ROUGE and ChatGPT performance as their positive control models (48.6% and 50.8% win rates for LLaMA-7B and FLAN-T5-XXL, respectively). But we are most interested in generalization to unseen tasks, as measured by the other two splits. On **Unseen** prompts within the Alpaca+ distribution, we again see competitive performance: 49.7% (LLaMA) and 46.2% (FLAN-T5) win rates against the positive controls. It is on the most challenging OOD **Human** split where we see slight drops in win rate to 45.8% (LLaMA) and 42.5% (FLAN-T5), though these numbers are still quite competitive with the positive control. Finally, gist compression is vastly superior to discrete compression; the TF-IDF models in Table 1 only marginally outperform the negative control models across the board.

Table 2 shows the human evaluation results on the Human validation split, comparing the single gist token models to the positive control. Overall, human annotators agree with ChatGPT, with average win rates of 52.3% (vs. 48.0%) for LLaMA-7B and 40.6% (vs. 42.0%) for FLAN-T5-XXL. Importantly, this agreement persists at the level of individual responses. The average pairwise Cohen's κ among human annotators is .24 for LLaMA-7B and .33 for FLAN-T5-XXL. Because humans will often arbitrarily choose one response over another even for samples of equal quality, these numbers

4.3 评估

我们的评估采用了提提示词来进行压缩。自动化指标与人工智能及人工评估方法的结合辅助式评估：

ROUGE-L指标。我们首先采用ROUGE-L指标，这是一种简单的词汇重叠统计方法 [20],，曾在以往的开放式指令微调研究中被使用 [37, 38]。除“人类”数据集组外，其他数据集组的参考标准均为text-davinci-{001,003}系列模型生成的文本，而“人类”数据集组则采用公认的黄金标准人类参考文本。

ChatGPT。 接下来，我们使用ChatGPT-3.5 [22] 来将我们所开发模型的输出结果与阳性对照进行对比。尽管这一评估指标并不完美，但它相比人类参与的实验能够实现更快、更低的成本，同时还能提供比ROUGE-L指标更具意义的语义信号。近期的一些研究发现在文本标注与评估 [12, 17, 35] 领域，ChatGPT的表现已接近人类水平；此外，针对最新推出的语言模型也进行了类似的基于模型的评估 [5, 11, 31]。

具体而言，给定一项任务 t、输入 x以及来自两个仅被标识为助手A和助手B的模型（y_1， y_2）的输出后，ChatGPT需要判断哪个助手的回答更优，并以思维链推理的方式 [39]阐述其判断依据。如果这两个模型产生的输出相同，或者表现同样糟糕，ChatGPT可以判定为平局。我们在ChatGPT的提示词中提供了期望的输出示例，同时为了消除顺序带来的影响，每次查询时都会随机调整两个模型的呈现顺序。给ChatGPT的完整提示词及评估细节详见附录C。通过这些输出，我们可以衡量某个模型相对于阳性对照的胜率：50%的胜率意味着该模型的质量与不进行提示词压缩的模型相当。

人类评估。最后，在使用ChatGPT完成原型设计后，我们会挑选出最佳的要点摘要压缩模型，并从人类验证数据集中的252个样本中随机抽取100个进行人类评估。针对这100个样本，我们从Prolific招募了3名来自美国或英国、英语流利的标注员，要求他们按照与上述ChatGPT评估相同的标准对模型输出进行评分（完整细节及标注界面详见附录D）。唯一的区别在于，当标注员因领域知识不足而无法准确判断响应内容时——例如遇到编程类问题——他们可以选择“我不知道”；在分析过程中我们会剔除这些响应（占比~10%）。通过这项人类评估，我们不仅旨在检验最终模型的性能，还要验证ChatGPT是否可以作为该任务中人类标注的可靠替代方案。

5个结果

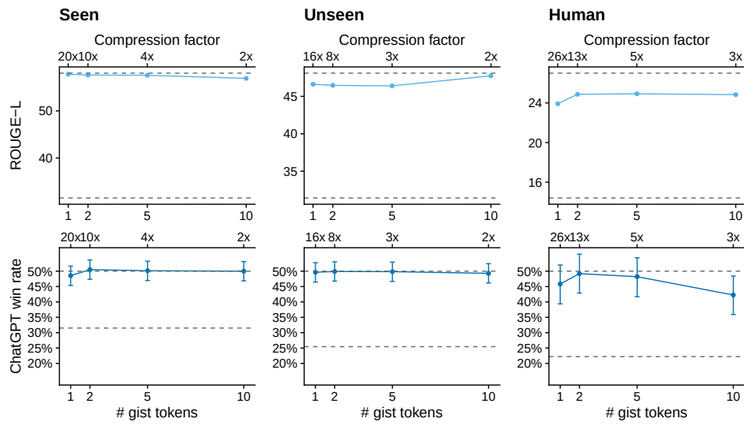
图3展示了针对LLaMA-7B模型、FLAN-T5-XXL模型，在不同数量摘要标记条件下的ROUGE-L指标及ChatGPT评估结果。总体而言，这些模型对摘要标记的数量较为不敏感 k：将提示词压缩为单个标记前缀并不会导致其性能显著劣于使用更长前缀的情况。实际上，在某些情况下过多的摘要标记反而会损害性能（例如LLaMA-7B模型使用10个摘要标记时），这可能是由于过大的容量使得模型更容易过拟合训练数据分布。因此，在本文的后续实验中我们均采用单个摘要标记的模型，并在表1中列出了该单标记模型的具体数值，同时给出了正面评价、负面评价以及基于TF-IDF算法的基准结果。

On **Seen** instructions, gist models attain near-identical ROUGE and ChatGPT performance as their positive control models (48.6% and 50.8% win rates for LLaMA-7B and FLAN-T5-XXL, respectively). But we are most interested in generalization to unseen tasks, as measured by the other two splits. On **Unseen** prompts within the Alpaca+ distribution, we again see competitive performance: 49.7% (LLaMA) and 46.2% (FLAN-T5) win rates against the positive controls. It is on the most challenging OOD **Human** split where we see slight drops in win rate to 45.8% (LLaMA) and 42.5% (FLAN-T5), though these numbers are still quite competitive with the positive control. Finally, gist compression is vastly superior to discrete compression; the TF-IDF models in Table 1 only marginally outperform the negative control models across the board.

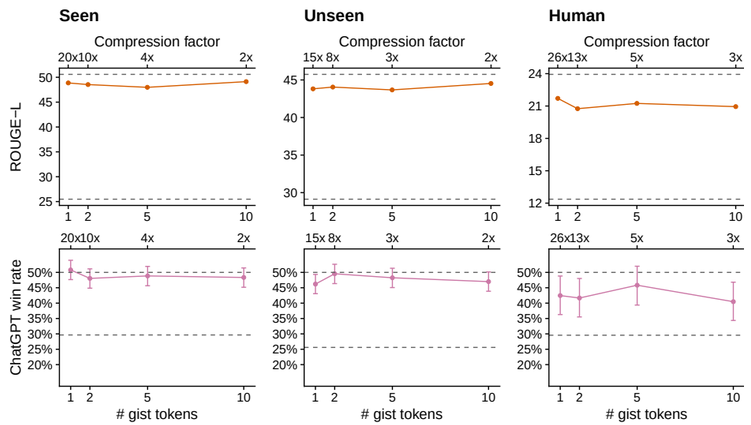
表2展示了在人类验证数据集上开展的人类评估结果，这些结果对比了仅使用要点摘要格式的提示词模型与阳性对照组的性能。总体而言，人类注释者对ChatGPT的评分较为一致，其平均胜率为52.3%（而LLaMA-7B的胜率为48.0%，FLAN-T5-XXL的胜率为40.6%）。更值得注意的是，这种一致性在单个响应层面也同样存在。人类注释者之间的平均成对科恩系数分别为：LLaMA-7B为0.24，FLAN-T5-XXL为0.33。由于即使对于质量相同的样本，人类也常常会随意选择某一个响应，因此这些数值

Table 1: **Results for single gist tokens.** ROUGE-L and ChatGPT scores for Gist, TF-IDF, and positive/negative controls. Parentheses are scores normalized between positive/negative controls.

		Seen		Unseen		Human	
Model		ROUGE-L	ChatGPT %	ROUGE-L	ChatGPT %	ROUGE-L	ChatGPT %
LLaMA-7B	Pos	58.0 (100)	50.0 (100)	48.1 (100)	50.0 (100)	27.0 (100)	50.0 (100)
	Gist	57.8 (99.2)	48.6 (92.4)	46.6 (91.0)	49.7 (98.8)	23.9 (75.4)	45.8 (84.9)
	TF-IDF	38.1 (24.5)	34.5 (16.2)	34.0 (15.6)	29.3 (15.9)	16.5 (16.7)	24.6 (8.6)
	Neg	31.5 (0)	31.5 (0)	31.4 (0)	25.4 (0)	14.4 (0)	22.2 (0)
FLAN-T5-XXL	Pos	50.6 (100)	50.0 (100)	45.7 (100)	50.0 (100)	23.9 (100)	50.0 (100)
	Gist	48.9 (93.2)	50.8 (103.9)	43.8 (88.6)	46.2 (84.4)	21.7 (80.9)	42.5 (63.2)
	TF-IDF	32.0 (25.9)	35.9 (30.5)	34.3 (31.3)	31.0 (22.1)	13.5 (9.6)	28.4 (-5.9)
	Neg	25.5 (0)	29.7 (0)	29.1 (0)	25.6 (0)	12.4 (0)	29.6 (0)



(a) **LLaMA-7B**

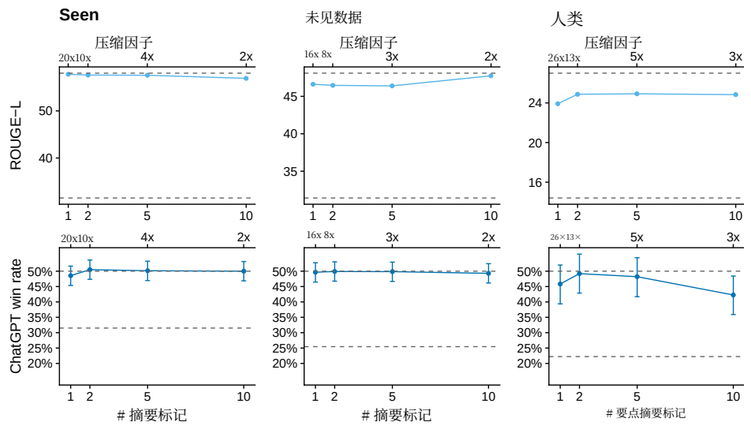


(b) **FLAN-T5-XXL**

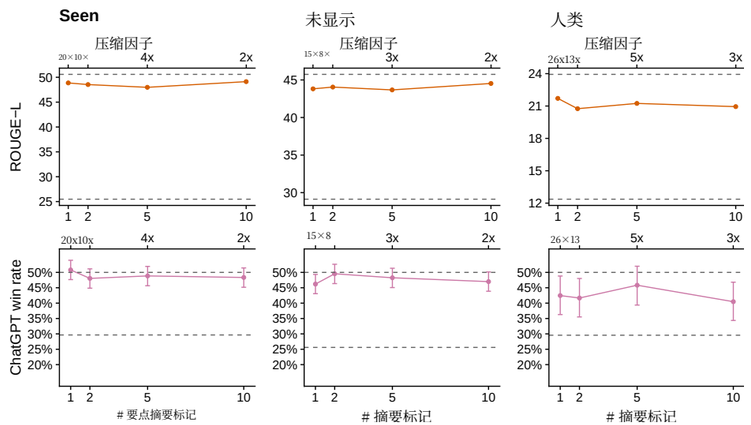
Figure 3: **Varying the number of gist tokens**. ROUGE-L and ChatGPT scores for (a) **LLaMA-7B** and (b) **FLAN-T5-XXL** for different gist tokens. Dashed lines indicate positive and negative control performance. Error bars are 95% exact binomial confidence intervals, splitting ties equally between models [10] and rounding down in favor of the positive control in case of an odd number of ties. Compression factors are calculated by computing the average token length of the validation split and dividing by the number of gist tokens.

表1：**单个摘要标记对应的测试结果**。针对摘要标记、TF-IDF算法以及正面评价/负面评价对照组的ROUGE-L指标与ChatGPT评分。括号内的数值为在正面评价/负面评价对照组之间标准化后的得分。

		Seen		未见过		人类	
模型		ROUGE-L	ChatGPT %	ROUGE-L	ChatGPT %	ROUGE-L	ChatGPT %
LLaMA-系列 7B	Pos	58.0（100）	50.0（100）	48.1（100）	50.0（100）	27.0（100）	50.0（100）
	Gist	57.8（99.2）	48.6（92.4）	46.6（91.0）	49.7（98.8）	23.9（75.4）	45.8（84.9）
	TF-IDF算法	38.1（24.5）	34.5（16.2）	34.0（15.6）	29.3（15.9）	16.5（16.7）	24.6（8.6）
	Neg	31.5（0）	31.5（0）	31.4（0）	25.4（0）	14.4（0）	22.2（0）
FLAN-模型 T5-XXL模型	Pos	50.6（100）	50.0（100）	45.7（100）	50.0（100）	23.9（100）	50.0（100）
	Gist	48.9（93.2）	50.8（103.9）	43.8（88.6）	46.2（84.4）	21.7（80.9）	42.5（63.2）
	TF-IDF算法	32.0（25.9）	35.9（30.5）	34.3（31.3）	31.0（22.1）	13.5（9.6）	28.4（-5.9）
	Neg	25.5（0）	29.7（0）	29.1（0）	25.6（0）	12.4（0）	29.6（0）



(a) **LLaMA-7B模型**



(b) **FLAN-T5-XXL模型**

图3：**不同数量的摘要标记对性能的影响**。展示了(a) **LLaMA-7B模型**与(b) **FLAN-T5-XXL模型**在采用不同数量摘要标记时的ROUGE-L指标与ChatGPT评分情况。虚线分别代表阳性对照与负面评价下的表现。误差条为95%精确二项式置信区间，当多个模型得分相同时会平均分配平局结果；若平局数为奇数，则优先将平局结果归入阳性对照一方。压缩因子则是通过计算验证集样本的平均Token长度，再除以摘要标记的数量得出的。

Table 2: **Human evaluation results.** Win rate and inter-annotator agreement of single token gist models over positive control according to 3 Human annotators (H1–H3), their average, and ChatGPT (95% confidence intervals in parentheses), for 100 out of 252 examples in the Human validation split.

	Gist Win % over Pos					Agreement (Cohen's κ)	
Model	H1	H2	H3	**Human (H1–H3)**	**ChatGPT**	**Human**	**ChatGPT**
LLaMA-7B	51.1	44.5	59.8	52.3 (46.1, 58.4)	48.0 (38.0, 58.2)	.24	.29
FLAN-T5-XXL	43.0	41.9	37.2	40.6 (34.6, 46.8)	42.0 (32.2, 52.3)	.33	.29

are fairly low; however, ChatGPT shows similar levels of agreement, with average κ across each of the 3 human annotators at .29 for both models. These results, paired with the similar overall win rates, show that using ChatGPT is similar to simply recruiting an additional human annotator, and corroborates the broader results in Figure 3. See Appendix D for more human evaluation results, including a breakdown of agreement across annotators, and Appendix G for examples of instructions, model outputs, and human/ChatGPT judgments in the Human validation split.

Since our aim is having the gist models mimic the original models, one might ask how often the gist model is identical to the positive control. Figure A.3 in Appendix E shows how often this happens: for **Seen** tasks (but unseen inputs), the gist model outputs exactly match the positive control nearly 50% of the time. This drops to ~20–25% for **Unseen** tasks and ~10% for the OOD **Human** tasks.

Overall, our results show that gist models can reliably compress prompts, even to some degree those that lie outside the training distribution, especially for decoder-only LMs (LLaMA). Encoder-decoder LMs like FLAN-T5 show slightly worse OOD performance; one possible reason is that gist masking inhibits the bidirectional attention flow in the encoder, which is more challenging to adjust to than simply limiting the history of a decoder LM. Future work will be needed to investigate this hypothesis.

5.1 Failure cases

While the responses from the gist models are often indistinguishable from the positive control, we nevertheless do see failure cases where the gist token models do not perfectly compress the prompt. We give two examples where this occurs in the Human validation split: first, when the instruction contains **specific details**, e.g. phrases that must be copied verbatim into the output (in the snippets below, the names of Assistants A/B are hidden to ChatGPT):

```
Instruction: Give the news title a category. Pick a category from the list of News &
Buzz, Travel, Style, Arts & Culture, Politics, Tech, and Science & Health. <G1>
Input: The #Banksy Exhibit in Cambridge, MA is absolutely terrific.
Assistant A (FLAN-T5-XXL gist): Culture
Assistant B (pos control): Arts & Culture

ChatGPT choice: B
ChatGPT reason: The news title is about an art exhibit, so Assistant B's response of
'Arts & Culture' is more accurate and specific than Assistant A's 'Culture'.
```

We also see more cases of **runaway generations** that do not occur with the positive control model:

```
Instruction: Provide an example of how a table of contents can be generated automatically
in a LATEX document. Make sure your example contains the necessary commands. <G1>
Assistant A (pos control): \tableofcontents \chapter{Introduction} \chapter{Theory}
\chapter {Experiments} \chapter{Conclusion}
Assistant B (LLaMA-7B gist): \tableofcontents \chapter{Introduction} \section{The Basics}
\subsection {The Basics} \subsubsection{The Basics} \paragraph{The Basics}
\subparagraph {The Basics} [repeats for hundreds of tokens] \subparagraph{The Basics}

ChatGPT choice: A
ChatGPT reason: Assistant A provides a simple and correct example of generating a table
of contents in LaTeX. Assistant B's example is unnecessarily long and does not follow
the standard structure of a table of contents.
```

While it is unclear why only the gist models exhibit this behavior, these issues can likely be mitigated with more careful sampling techniques.

表2：**人类评估结果**。基于人类验证数据集中的252个样本中的100个样本，展示了单词短语摘要模型相对于阳性对照组的胜率与注释者间一致性，数据来源于3位人类注释者（H1–H3），同时给出了这些指标的平均值以及ChatGPT的相关数值（括号内为95%置信区间）。

	针对阳性对照组的要点摘要模型胜率百分比阳性对照组					一致性（Cohen's κ）	
模型	H1	H2	H3	**人类注释者（H1–H3）**	**ChatGPT**	**人类**	**ChatGPT**
LLaMA-7B模型	51.1	44.5	59.8	52.3（46.1，58.4）	48.0（38.0，58.2）	.24	.29
FLAN-T5-XXL模型	43.0	41.9	37.2	40.6（34.6，46.8）	42.0（32.2，52.3）	.33	.29

这些数值相对较低；不过ChatGPT的匹配程度也处于相似水平，两种模型在3位人类注释者各自的 κ 平均值均为0.29。结合两者相近的整体胜率，这些结果表明使用ChatGPT与额外招募一名人类注释者效果类似，同时也印证了图3中所呈现的更广泛研究结果。如需更多人类评估结果，包括不同注释者之间的匹配度细分数据，请参阅附录D；而人类验证数据集中指令、模型输出以及人类/ChatGPT判断结果的示例则可见于附录G。

由于我们的目标是让要点摘要模型模仿原始模型，因此有人可能会疑问：要点摘要模型与阳性对照的匹配频率究竟有多高？附录E中的图A.3展示了这一情况的出现频率：在**已知**任务中（但输入数据为未知），要点摘要模型的输出与阳性对照几乎有50%的时间完全一致。而在**未知**任务中，这一比例降至 ~20%–25%，对于OOD **人类**任务，则仅为 ~10%。

总体而言，我们的研究结果表明，要点摘要模型能够可靠地压缩提示词，即便是对那些处于训练数据分布之外的提示词也能在一定程度上实现压缩，这一效果在仅包含解码器的语言模型（如LLaMA）中尤为明显。而像FLAN-T5这类兼具编码器与解码器的语言模型在处理外分布数据的性能则稍差一些；其原因之一可能是摘要单元掩码会阻碍编码器中的双向注意力流动，相比单纯限制解码器语言模型的历史上下文，这种调整难度更大。未来需要进一步的研究来验证这一假设。

5.1 失败案例

虽然要点摘要模型生成的响应往往与阳性对照几乎无法区分，但我们仍然观察到了部分失败案例，即这些要点摘要令牌模型未能完美压缩提示词。我们在人类验证数据集中列举了两个此类案例：首先，当指令中包含**具体细节**时，也就是那些必须原封不动地复制到输出结果中的短语（在以下的示例中，助手A/B的名称已对ChatGPT进行隐藏处理）：

```
说明：为新闻标题指定一个类别。请从新闻与趣闻类、旅游类、时尚类、艺术与文化类、政治
类、科技类以及科学与健康类中选择合适的类别。 <G1>输入内容：位于马萨诸塞州剑桥市的#
班克斯展览实在精彩极了。助手A(FLAN-T5-XXL要点摘要)：文化类助手B(正面评价控制)：艺
术与文化类

ChatGPT的选择：B
ChatGPT的推理原因：这条新闻标题是关于一场艺术展的，所以助手B'的回应是
'Arts与文化'比助手A'更准确、更具体'文化'。
```

我们还观察到了更多**失控生成**的现象，而这类现象在正控制模型中是不会出现的：

```
Instruction: Provide an example of how a table of contents can be generated automatically
in a LATEX document. Make sure your example contains the necessary commands. <G1>
Assistant A (pos control): \tableofcontents \chapter{Introduction} \chapter{Theory}
\chapter {Experiments} \chapter{Conclusion}
Assistant B (LLaMA-7B gist): \tableofcontents \chapter{Introduction} \section{The Basics}
\subsection {The Basics} \subsubsection{The Basics} \paragraph{The Basics}
\subparagraph {The Basics} [repeats for hundreds of tokens] \subparagraph{The Basics}

ChatGPT选择：A
ChatGPT的说明：助手A给出了一个简单且正确的LATEX目录生成示例。助手B其给出的示例过长，
且并未遵循目录的标准结构。
```

尽管目前尚不清楚为何仅有要点摘要模型会出现这种行为，但通过采用更精细的采样技术，很可能会缓解这些问题。

Table 3: **Gist efficiency improvements**. For different caching strategies (**None**, **Instruction**, **Gist**), we record CUDA wall time and GFLOPs (± std dev). Then we report the absolute/relative improvement of **Gist Caching** over these alternative strategies, with 95% confidence intervals in parentheses.

Model	Metric	Caching Strategy None	Instruction[i]	Gist[ii]	Absolute/Relative Δ vs. None	vs. Instruction
LLaMA-7B	CUDA time (ms) ↓	23.4 ± 6.88	22.1 ± 6.58	**21.8 ± 6.55**	1.60 (1.29, 1.90) 6.8% (5.5, 8.1)	.221 (.140, .302) 1.0% (.63, 1.4)
	GFLOPs ↓	915 ± 936	553 ± 900	**552 ± 899**	362 (337, 387) 40% (37, 42)	.607 (.448, .766) .11% (.08, .14)
FLAN-T5-XXL	CUDA time (ms) ↓	31.0 ± 5.31	N/A	**29.7 ± 5.07**	1.30 (1.10, 1.51) 4.2% (3.6, 4.9)	N/A
	GFLOPs ↓	716 ± 717	N/A	**427 ± 684**	289 (268, 310) 40% (37, 43)	N/A

[i]Average KV Cache Length = 26 [ii]Average KV Cache Length = 1

6 Compute, Memory, and Storage Efficiency

Finally, we return to one of the central motivations of this paper: what kind of efficiency gains does gisting enable? To answer this question, we compare the compute requirements (CUDA wall time and FLOPs) during inference with the single-token gist models using different caching strategies:

1. **No caching**: just encoding the full prompt t.
2. **Instruction caching**: caching the activations of the uncompressed instruction t (keys and values for all layers) into what is called the **KV cache**. This is the most common caching behavior for Transformer inference [4, 26] and is supported in libraries like Hugging Face Transformers [41]. However, it is only applicable to *decoder-only* models, since in models with bidirectional encoders like T5, the instruction representations t depend on the input x.
3. **Gist caching**: Compressing the prompt into the gist prefix $G(t)$.

Table 3 displays the results of profiling a single forward pass through the model (i.e. one step of decoding with a single input token) with PyTorch [24] 2.0, averaged across the 252 Human instructions. Gist caching improves significantly over unoptimized models, with 40% FLOPs savings and 4-7% lower wall time for both models. Note that at these (relatively) small scales, the wall time improvements are smaller than the FLOPs reductions because much of the inference latency is caused by moving tensors from high-bandwidth memory (HBM) to the chip compute cores, i.e. what Pope et al. [26] call the "memory time". Larger sequence lengths and batch sizes will lead to additional speedups, as the overall latency becomes dominated by the actual matrix computations.

For LLaMA-7B, the picture is more nuanced when compared to caching the full instruction. Compute improvements of gist caching are smaller: a negligible decrease in FLOPs (0.11%) and a modest 1% speedup in wall time. This is because the FLOPs required for a Transformer forward pass is dominated by processing of the new input tokens, rather than self-attention with the KV cache. For example, a forward pass through LLaMA-7B with a single input token and a *2000-length* KV cache is only ~10% more expensive than the same forward pass with no KV cache—see Appendix F for more details. Nevertheless, this small decrease in FLOPs leads to a disproportionate decrease in wall time (1%), likely because the self-attention computations are slower relative to their FLOPs contribution.

At large scales and with heavily reused prompts, a 1% latency speedup can still accumulate into significant cost savings over time. More importantly, however, there are key benefits of gist caching over instruction caching besides latency: compressing 26 tokens into 1 gives more space in the input context window, which is bounded by absolute position embeddings or GPU VRAM. For example, for LLaMA-7B, each token in the KV cache requires 1.05 MB storage.[3] While the total contribution of the KV cache relative to the memory needed for LLaMA-7B inference is negligible at the prompt lengths we tested, an increasingly common scenario is developers caching many prompts across a large number of users, where storage costs quickly add up. In these scenarios, gisting allows caching of up to **26x** more prompts than full instruction caching, using the same amount of storage!

[3]4 (fp32 bytes) × 2 (keys+values) × 32 (num layers) × 32 (num attn heads) × 128 (head dim) = 1.05 MB.

表3：**要点摘要的效率提升情况**。针对不同的缓存策略（**无**、**指令引导型**、**要点摘要型**），我们记录了CUDA计算耗时与GFLOPs运算量（± 标准差）。随后我们会报告绝对/相对的**要点缓存**相较于这些其他策略的提升幅度，并在括号内标注95%置信区间。

模型	指标	缓存策略 None	指令[i]	要点摘要[ii]	绝对路径/相对路径 Δ 与“无”相比	与“指令引导”相比
LLaMA-系列 7B	CUDA 时间（毫秒）↓	23.4 ± 6.88	22.1 ± 6.58	**21.8 ± 6.55**	1.60（1.29，1.90） 6.8%（5.5，8.1）	.221（.140，.302） 1.0%（.63，1.4）
	GFLOPs：↓	915 ± 936	553 ± 900	**552 ± 899**	362（337，387） 40%（37，42）	0.607（0.448，0.766） 0.11%（0.08，0.14）
FLAN-模型 T5-XXL模型	CUDA 时间（毫秒）↓	31.0 ± 5.31	N/A	**29.7 ± 5.07**	1.30（1.10，1.51） 4.2%（3.6，4.9）	N/A
	GFLOPs：↓	716 ± 717	N/A	**427 ± 684**	289（268，310） 40%（37，43）	N/A

[i]KV缓存的平均长度 = 26 ii = 1KV缓存的平均长度

6 计算、内存与存储效率

最后，我们回到本文的核心出发点之一：摘要化技术能带来怎样的效率提升？为回答这一问题，我们对比了在使用不同缓存策略的单提示词要点摘要模型进行推理时的计算需求（CUDA计算耗时与浮点运算次数）。

1. **不进行缓存**：仅对完整的提示词 t进行编码。2. **指令缓存**：将未压缩指令 t 的激活值（所有层的键和值）缓存到所谓的**KV缓存**中。这是<transformer inference> [4, 26] 中最常见的缓存方式，也被Hugging Face Transformers [41]等库所支持。不过，这种方式仅适用于仅解码器类型的模型，因为在T5这类具有双向编码器的模型中，指令表示 t 取决于输入 x。
3. **要点缓存**：将提示词压缩为要点摘要前缀 $G(t)$。

表3展示了使用PyTorch [24] 2.0对模型进行单次前向传播（即针对单个输入标记执行一步解码操作）的性能分析结果，该数据是基于252条人类指令集计算得出的平均值。与未经优化的模型相比，要点缓存带来了显著提升：两种模型的浮点运算次数均减少了40%，且运行时间分别降低了4%-7%。需要注意的是，在这种（相对）较小的规模下，运行时间的缩减幅度要小于浮点运算次数的减少幅度，因为推理延迟的主要来源是将张量从高带宽内存（HBM）传输到芯片计算核心，也就是Pope等人 [26] 所称的“内存时间”。当序列长度和批量大小增大时，由于整体延迟将主要由实际的矩阵运算决定，性能提升还会更加明显。

对于LLaMA-7B模型而言，与缓存完整指令相比，情况要复杂一些。要点缓存带来的计算性能提升较为有限：浮点运算次数仅出现可忽略的0.11%下降，而整体运行时间则仅有1%的轻微提升。这是因为Transformer模型进行前向传播时所需的浮点运算次数主要取决于新输入令牌的处理，而非与KV缓存相关的自注意力机制运算。例如，当输入仅包含一个令牌且KV缓存的长度为*2000*时，LLaMA-7B的此次前向传播所需成本仅比没有KV缓存的相同操作高出 ~10%——更多细节请参见附录F。尽管如此，浮点运算次数的这种微小下降依然导致了运行时间1%的显著缩短，这很可能是因为自注意力机制的运算速度相对其对应的浮点运算贡献而言较慢所致。

At large scales and with heavily reused prompts, a 1% latency speedup can still accumulate into significant cost savings over time. More importantly, however, there are key benefits of gist caching over instruction caching besides latency: compressing 26 tokens into 1 gives more space in the input context window, which is bounded by absolute position embeddings or GPU VRAM. For example, for LLaMA-7B, each token in the KV cache requires 1.05 MB storage.[3] While the total contribution of the KV cache relative to the memory needed for LLaMA-7B inference is negligible at the prompt lengths we tested, an increasingly common scenario is developers caching many prompts across a large number of users, where storage costs quickly add up. In these scenarios, gisting allows caching of up to **26x** more prompts than full instruction caching, using the same amount of storage!

[3]4 (fp32字节) × 2 (键值)+值） × 32 （层数） × 32 （注意力头数） × 128（头部维度） = 1.05 MB。

7 Additional Related Work

Gisting builds upon past work in (parameter-efficient) instruction finetuning and context distillation, as discussed in Section 2. Here we will outline some additional connections to related work:

Adapting LMs without Backprop. As mentioned in Section 2, gisting can be viewed as a way to adapt LMs *without* gradient descent, by predicting the the prefix ($\approx$ parameters) of the adapted model. Similarly, HyperTuning [25] predicts the prefix of a model for a task using (input, output) pairs for that task. If HyperTuning is a "few-shot" adaptation method, predicting the prefix from few-shot examples, then gisting can be seen as a "zero-shot" version, predicting the prefix from the language instruction alone. Gisting also has the additional benefit over HyperTuning of being conceptually simpler: instead of training a separate LM to predict the prefix, the LM itself is used as the HyperNetwork [13], with only a tiny change to the attention masks needed for prompt compression.

Compression and memory in transformers. The idea of "compressing" prompts is closely related to previous attempts at storing past representations to improve memory and long-range sequence modeling in Transformers [9, 21, 27, 42, 43]. In particular, the Compressive Transformer [27] compresses transformer activations into a smaller *compressed memory* using a learned convolutional operator. Gisting can be seen as a variant of the Compressive Transformer with 3 key differences. First, the compression function is not a separately learned function, but the LM's own self-attention mechanism, controlled by an input-dependent gist token. Second, the compression function is learned jointly with instruction finetuning via the standard language modeling loss, not a specialized auxiliary reconstruction loss as in [27]. Finally, our task of interest is not long-range sequence modeling, but caching and reusing instruction following prompts for efficiency reasons.

Sparse attention mechanisms. By restricting attention masks, gisting draws inspiration from efficient/sparse attention methods in Transformers (see [32] for review). For example, some sliding window attention mechanisms [2, 6] may remove the need to keep the entire KV cache around during decoding, but these more general methods are not optimized for caching arbitrary parts of the input sequence of varying length, which prompt compression demands. In light of this, gisting can be viewed as an input-dependent sparse attention mechanism specifically aimed at improving efficiency of the prompting workflow now commonly used in LMs.

8 Discussion and Limitations

In this paper we presented gisting, a framework for prompt compression in LMs, and a simple way of implementing gist models by modifying Transformer attention masks that incurs no additional cost over standard instruction finetuning. Gisting can be seen either as a modified form of instruction finetuning or a method for (meta-)context distillation of an LM. Gist models can compress unseen OOD prompts up to 26x while maintaining output quality, resulting in up to 40% FLOPs reduction and 4.2% wall clock speedups over unoptimized models, and enabling new methods for prompt caching in encoder-decoder models. While wall-time improvements for decoder-only LMs are smaller, gisting nevertheless enables caching 1 *order of magnitude* (26x) more prompts relative to full instructions.

Gisting is a promising method for improving LM efficiency, but carries some limitations. While gisting seems to succeed in capturing the "gist" of instructions (hence the name), achieving such compression necessarily results in some loss of nuance of the original instruction; Secton 5.1 illustrates a concrete failure case we observed. Since the behavior of LMs on edge cases is already not well understood, it is *especially* important for practitioners to carefully evaluate whether the compute/accuracy tradeoff of gisting is sufficiently safe and robust for their use case, *before* deployment.

Nevertheless, we believe gisting opens several interesting directions for future work. First, the masking method presented here can be easily integrated into existing instruction finetuning workflows, but another exciting approach is to retrofit an existing, frozen LM by training a *smaller* model to compress prompts, if finetuning the larger LM is inconvenient. Second, the largest efficiency gains from gisting will result from compressing very long prompts, for example k-shot prompts for large k that may even exceed a single context window. Finally, compression performance can likely be improved through "gist pretraining": first learning to compress arbitrary spans of natural language, before then learning prompt compression. Such objectives could be devised by inserting gist tokens into other pretraining objectives, perhaps during language modeling or T5's span corruption objective.

7项其他相关研究

如第2节所述，摘要化技术是在（参数效率高的）指令微调与上下文蒸馏的先前研究基础之上发展而来的。在此，我们将进一步阐述其与相关研究的更多关联。

Adapting LMs without Backprop. As mentioned in Section 2, gisting can be viewed as a way to adapt LMs *without* gradient descent, by predicting the the prefix ($\approx$ parameters) of the adapted model. Similarly, HyperTuning [25] predicts the prefix of a model for a task using (input, output) pairs for that task. If HyperTuning is a "few-shot" adaptation method, predicting the prefix from few-shot examples, then gisting can be seen as a "zero-shot" version, predicting the prefix from the language instruction alone. Gisting also has the additional benefit over HyperTuning of being conceptually simpler: instead of training a separate LM to predict the prefix, the LM itself is used as the HyperNetwork [13], with only a tiny change to the attention masks needed for prompt compression.

Compression and memory in transformers. The idea of "compressing" prompts is closely related to previous attempts at storing past representations to improve memory and long-range sequence modeling in Transformers [9, 21, 27, 42, 43]. In particular, the Compressive Transformer [27] compresses transformer activations into a smaller *compressed memory* using a learned convolutional operator. Gisting can be seen as a variant of the Compressive Transformer with 3 key differences. First, the compression function is not a separately learned function, but the LM's own self-attention mechanism, controlled by an input-dependent gist token. Second, the compression function is learned jointly with instruction finetuning via the standard language modeling loss, not a specialized auxiliary reconstruction loss as in [27]. Finally, our task of interest is not long-range sequence modeling, but caching and reusing instruction following prompts for efficiency reasons.

稀疏注意力机制。通过限制注意力掩码，摘要化技术借鉴了Transformer模型中高效/稀疏的注意力处理方法（相关综述见 [32] ）。例如，某些滑动窗口注意力机制 [2, 6]可以消除在解码过程中需要保留完整KV缓存的需求，但这些更通用的方法并未针对提示词压缩所要求的、对不同长度的输入序列任意部分进行缓存的需求进行优化。有鉴于此，摘要化技术可被视为一种依赖输入的稀疏注意力机制，专门用于提升当前在语言模型中广泛使用的提示词处理流程的效率。

8 讨论与局限性

In this paper we presented gisting, a framework for prompt compression in LMs, and a simple way of implementing gist models by modifying Transformer attention masks that incurs no additional cost over standard instruction finetuning. Gisting can be seen either as a modified form of instruction finetuning or a method for (meta-)context distillation of an LM. Gist models can compress unseen OOD prompts up to 26x while maintaining output quality, resulting in up to 40% FLOPs reduction and 4.2% wall clock speedups over unoptimized models, and enabling new methods for prompt caching in encoder-decoder models. While wall-time improvements for decoder-only LMs are smaller, gisting nevertheless enables caching 1 *order of magnitude* (26x) more prompts relative to full instructions.

摘要化技术是一种有望提升语言模型效率的方法，但也存在一些局限性。尽管该技术似乎能够捕捉到指令的“要点摘要”（也因此得名），但这样的压缩处理必然会导致原始指令的部分细节丢失；第5.1节展示了我们观察到的一个具体失败案例。由于目前人们对语言模型在边缘场景下的表现仍了解不足，因此对于从业者而言，在<span style='id':'1'>尤其部署之前，仔细评估摘要化技术所带来的计算成本与准确率之间的权衡是否足够安全且稳健，显得十分<span style='id':'3'>重要。

Nevertheless, we believe gisting opens several interesting directions for future work. First, the masking method presented here can be easily integrated into existing instruction finetuning workflows, but another exciting approach is to retrofit an existing, frozen LM by training a *smaller* model to compress prompts, if finetuning the larger LM is inconvenient. Second, the largest efficiency gains from gisting will result from compressing very long prompts, for example k-shot prompts for large k that may even exceed a single context window. Finally, compression performance can likely be improved through "gist pretraining": first learning to compress arbitrary spans of natural language, before then learning prompt compression. Such objectives could be devised by inserting gist tokens into other pretraining objectives, perhaps during language modeling or T5's span corruption objective.

Acknowledgments and Disclosure of Funding

We thank the Stanford Alpaca team, especially Xuechen Li, with assistance with Alpaca data and finetuning, and Gabriel Poesia for the codebase used to collect human evaluations. Additional thanks to Xiuyu Li for help debugging and ensuring reproducibility of the open-source codebase. JM is supported by the Open Philanthropy AI Fellowship and XLL is supported by the Stanford Graduate Fellowship.

References

[1] A. Askell, Y. Bai, A. Chen, D. Drain, D. Ganguli, T. Henighan, A. Jones, N. Joseph, B. Mann, N. DasSarma, et al. A general language assistant as a laboratory for alignment. *arXiv preprint arXiv:2112.00861*, 2021.

[2] I. Beltagy, M. E. Peters, and A. Cohan. Longformer: The long-document transformer. *arXiv:2004.05150*, 2020.

[3] T. Brown, B. Mann, N. Ryder, M. Subbiah, J. D. Kaplan, P. Dhariwal, A. Neelakantan, P. Shyam, G. Sastry, A. Askell, et al. Language models are few-shot learners. *Advances in Neural Information Processing Systems*, 33:1877–1901, 2020.

[4] C. Chen. Transformer inference arithmetic. `https://kipp.ly/blog/transformer-inference-arithmetic/`, 2022.

[5] W.-L. Chiang, Z. Li, Z. Lin, Y. Sheng, Z. Wu, H. Zhang, L. Zheng, S. Zhuang, Y. Zhuang, J. E. Gonzalez, I. Stoica, and E. P. Xing. Vicuna: An open-source chatbot impressing GPT-4 with 90% ChatGPT quality. `https://vicuna.lmsys.org/`, 2023.

[6] R. Child, S. Gray, A. Radford, and I. Sutskever. Generating long sequences with sparse transformers. *arXiv preprint arXiv:1904.10509*, 2019.

[7] E. Choi, Y. Jo, J. Jang, and M. Seo. Prompt injection: Parameterization of fixed inputs. *arXiv preprint arXiv:2206.11349*, 2022.

[8] H. W. Chung, L. Hou, S. Longpre, B. Zoph, Y. Tay, W. Fedus, E. Li, X. Wang, M. Dehghani, S. Brahma, et al. Scaling instruction-finetuned language models. *arXiv preprint arXiv:2210.11416*, 2022.

[9] Z. Dai, Z. Yang, Y. Yang, J. G. Carbonell, Q. Le, and R. Salakhutdinov. Transformer-XL: Attentive language models beyond a fixed-length context. In *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics*, pages 2978–2988, 2019.

[10] W. J. Dixon and F. J. Massey Jr. *Introduction to statistical analysis.* McGraw-Hill, 1951.

[11] X. Geng, A. Gudibande, H. Liu, E. Wallace, P. Abbeel, S. Levine, and D. Song. Koala: A dialogue model for academic research. Blog post, April 2023. URL `https://bair.berkeley.edu/blog/2023/04/03/koala/`.

[12] F. Gilardi, M. Alizadeh, and M. Kubli. ChatGPT outperforms crowd-workers for text-annotation tasks. *arXiv preprint arXiv:2303.15056*, 2023.

[13] D. Ha, A. Dai, and Q. V. Le. HyperNetworks. In *International Conference on Learning Representations*, 2017.

[14] J. Hoffmann, S. Borgeaud, A. Mensch, E. Buchatskaya, T. Cai, E. Rutherford, D. d. L. Casas, L. A. Hendricks, J. Welbl, A. Clark, et al. Training compute-optimal large language models. *arXiv preprint arXiv:2203.15556*, 2022.

[15] N. Houlsby, A. Giurgiu, S. Jastrzebski, B. Morrone, Q. De Laroussilhe, A. Gesmundo, M. Attariyan, and S. Gelly. Parameter-efficient transfer learning for NLP. In *International Conference on Machine Learning*, pages 2790–2799. PMLR, 2019.

致谢与资金来源说明

我们感谢斯坦福大学Alpaca模型团队，尤其是在Alpaca数据提供及微调方面给予帮助的李学晨，同时也感谢Gabriel Poesia提供了用于收集人类评估数据的代码库。此外，还要感谢李秀玉在调试以及确保该开源代码库可复现性方面的协助。JM的研究工作得到了开放慈善人工智能奖学金的支持，而XLL则得益于斯坦福大学研究生奖学金的资助。

参考资料

[1] A. Askell, Y. Bai, A. Chen, D. Drain, D. Ganguli, T. Henighan, A. Jones, N. Joseph, B. Mann, N. DasSarma, et al. A general language assistant as a laboratory for alignment. *arXiv preprint arXiv:2112.00861*, 2021.

[2] I. Beltagy, M. E. Peters, and A. Cohan. Longformer: The long-document transformer. *arXiv:2004.05150*, 2020.

[3] T. Brown, B. Mann, N. Ryder, M. Subbiah, J. D. Kaplan, P. Dhariwal, A. Neelakantan, P. Shyam, G. Sastry, A. Askell, et al. Language models are few-shot learners. *Advances in Neural Information Processing Systems*, 33:1877–1901, 2020.

[4] C. Chen. Transformer inference arithmetic. `https://kipp.ly/blog/transformer-inference-arithmetic/`, 2022.

[5] W.-L. Chiang, Z. Li, Z. Lin, Y. Sheng, Z. Wu, H. Zhang, L. Zheng, S. Zhuang, Y. Zhuang, J. E. Gonzalez, I. Stoica, and E. P. Xing. Vicuna: An open-source chatbot impressing GPT-4 with 90% ChatGPT quality. `https://vicuna.lmsys.org/`, 2023.

[6] R. Child, S. Gray, A. Radford, and I. Sutskever. Generating long sequences with sparse transformers. *arXiv preprint arXiv:1904.10509*, 2019.

[7] E. Choi, Y. Jo, J. Jang, and M. Seo. Prompt injection: Parameterization of fixed inputs. *arXiv preprint arXiv:2206.11349*, 2022.

[8] H. W. Chung, L. Hou, S. Longpre, B. Zoph, Y. Tay, W. Fedus, E. Li, X. Wang, M. Dehghani, S. Brahma, et al. Scaling instruction-finetuned language models. *arXiv preprint arXiv:2210.11416*, 2022.

[9] Z. Dai, Z. Yang, Y. Yang, J. G. Carbonell, Q. Le, and R. Salakhutdinov. Transformer-XL: Attentive language models beyond a fixed-length context. In *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics*, pages 2978–2988, 2019.

[10] W. J. Dixon and F. J. Massey Jr. *Introduction to statistical analysis.* McGraw-Hill, 1951.

[11] X. Geng, A. Gudibande, H. Liu, E. Wallace, P. Abbeel, S. Levine, and D. Song. Koala: A dialogue model for academic research. Blog post, April 2023. URL `https://bair.berkeley.edu/blog/2023/04/03/koala/`.

[12] F. Gilardi, M. Alizadeh, and M. Kubli. ChatGPT outperforms crowd-workers for text-annotation tasks. *arXiv preprint arXiv:2303.15056*, 2023.

[13] D. Ha, A. Dai, and Q. V. Le. HyperNetworks. In *International Conference on Learning Representations*, 2017.

[14] J. Hoffmann, S. Borgeaud, A. Mensch, E. Buchatskaya, T. Cai, E. Rutherford, D. d. L. Casas, L. A. Hendricks, J. Welbl, A. Clark, et al. Training compute-optimal large language models. *arXiv preprint arXiv:2203.15556*, 2022.

[15] N. Houlsby, A. Giurgiu, S. Jastrzebski, B. Morrone, Q. De Laroussilhe, A. Gesmundo, M. Attariyan, and S. Gelly. Parameter-efficient transfer learning for NLP. In *International Conference on Machine Learning*, pages 2790–2799. PMLR, 2019.

[16] E. J. Hu, Y. Shen, P. Wallis, Z. Allen-Zhu, Y. Li, S. Wang, L. Wang, and W. Chen. LoRA: Low-rank adaptation of large language models. In *International Conference on Learning Representations*, 2022.

[17] F. Huang, H. Kwak, and J. An. Is ChatGPT better than human annotators? Potential and limitations of ChatGPT in explaining implicit hate speech. *arXiv preprint arXiv:2302.07736*, 2023.

[18] B. Lester, R. Al-Rfou, and N. Constant. The power of scale for parameter-efficient prompt tuning. In *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing*, pages 3045–3059, 2021.

[19] X. L. Li and P. Liang. Prefix-tuning: Optimizing continuous prompts for generation. In *Proceedings of the 59th Annual Meeting of the Association for Computational Linguistics and the 11th International Joint Conference on Natural Language Processing (Volume 1: Long Papers)*, pages 4582–4597, 2021.

[20] C.-Y. Lin. ROUGE: A package for automatic evaluation of summaries. In *Text summarization branches out*, pages 74–81, 2004.

[21] P. J. Liu, M. Saleh, E. Pot, B. Goodrich, R. Sepassi, L. Kaiser, and N. Shazeer. Generating Wikipedia by summarizing long sequences. In *International Conference on Learning Representations*, 2018.

[22] OpenAI. Introducing ChatGPT. `https://openai.com/blog/chatgpt`, 2022.

[23] L. Ouyang, J. Wu, X. Jiang, D. Almeida, C. Wainwright, P. Mishkin, C. Zhang, S. Agarwal, K. Slama, A. Ray, et al. Training language models to follow instructions with human feedback. In *Advances in Neural Information Processing Systems*, pages 27730–27744, 2022.

[24] A. Paszke, S. Gross, F. Massa, A. Lerer, J. Bradbury, G. Chanan, T. Killeen, Z. Lin, N. Gimelshein, L. Antiga, et al. PyTorch: An imperative style, high-performance deep learning library. *Advances in Neural Information Processing Systems*, 32, 2019.

[25] J. Phang, Y. Mao, P. He, and W. Chen. HyperTuning: Toward adapting large language models without back-propagation. *arXiv preprint arXiv:2211.12485*, 2022.

[26] R. Pope, S. Douglas, A. Chowdhery, J. Devlin, J. Bradbury, A. Levskaya, J. Heek, K. Xiao, S. Agrawal, and J. Dean. Efficiently scaling transformer inference. *arXiv preprint arXiv:2211.05102*, 2022.

[27] J. W. Rae, A. Potapenko, S. M. Jayakumar, C. Hillier, and T. P. Lillicrap. Compressive transformers for long-range sequence modelling. In *International Conference on Learning Representations*, 2020.

[28] C. Raffel, N. Shazeer, A. Roberts, K. Lee, S. Narang, M. Matena, Y. Zhou, W. Li, and P. J. Liu. Exploring the limits of transfer learning with a unified Text-to-Text Transformer. *The Journal of Machine Learning Research*, 21(1):5485–5551, 2020.

[29] J. Rasley, S. Rajbhandari, O. Ruwase, and Y. He. DeepSpeed: System optimizations enable training deep learning models with over 100 billion parameters. In *Proceedings of the 26th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining*, pages 3505–3506, 2020.

[30] C. Snell, D. Klein, and R. Zhong. Learning by distilling context. *arXiv preprint arXiv:2209.15189*, 2022.

[31] R. Taori, I. Gulrajani, T. Zhang, Y. Dubois, X. Li, C. Guestrin, P. Liang, and T. B. Hashimoto. Stanford Alpaca: An instruction-following LLaMA model. `https://github.com/tatsu-lab/stanford_alpaca`, 2023.

[32] Y. Tay, M. Dehghani, D. Bahri, and D. Metzler. Efficient transformers: A survey. *ACM Computing Surveys*, 55(6):1–28, 2022.

[16] E. J. Hu, Y. Shen, P. Wallis, Z. Allen-Zhu, Y. Li, S. Wang, L. Wang, and W. Chen. LoRA: Low-rank adaptation of large language models. In *International Conference on Learning Representations*, 2022.

[17] F. Huang, H. Kwak, and J. An. Is ChatGPT better than human annotators? Potential and limitations of ChatGPT in explaining implicit hate speech. *arXiv preprint arXiv:2302.07736*, 2023.

[18] B. Lester, R. Al-Rfou, and N. Constant. The power of scale for parameter-efficient prompt tuning. In *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing*, pages 3045–3059, 2021.

[19] X. L. Li and P. Liang. Prefix-tuning: Optimizing continuous prompts for generation. In *Proceedings of the 59th Annual Meeting of the Association for Computational Linguistics and the 11th International Joint Conference on Natural Language Processing (Volume 1: Long Papers)*, pages 4582–4597, 2021.

[20] C.-Y. Lin. ROUGE: A package for automatic evaluation of summaries. In *Text summarization branches out*, pages 74–81, 2004.

[21] P. J. Liu, M. Saleh, E. Pot, B. Goodrich, R. Sepassi, L. Kaiser, and N. Shazeer. Generating Wikipedia by summarizing long sequences. In *International Conference on Learning Representations*, 2018.

[22] OpenAI. Introducing ChatGPT. `https://openai.com/blog/chatgpt`, 2022.

[23] L. Ouyang, J. Wu, X. Jiang, D. Almeida, C. Wainwright, P. Mishkin, C. Zhang, S. Agarwal, K. Slama, A. Ray, et al. Training language models to follow instructions with human feedback. In *Advances in Neural Information Processing Systems*, pages 27730–27744, 2022.

[24] A. Paszke, S. Gross, F. Massa, A. Lerer, J. Bradbury, G. Chanan, T. Killeen, Z. Lin, N. Gimelshein, L. Antiga, et al. PyTorch: An imperative style, high-performance deep learning library. *Advances in Neural Information Processing Systems*, 32, 2019.

[25] J. Phang, Y. Mao, P. He, and W. Chen. HyperTuning: Toward adapting large language models without back-propagation. *arXiv preprint arXiv:2211.12485*, 2022.

[26] R. Pope, S. Douglas, A. Chowdhery, J. Devlin, J. Bradbury, A. Levskaya, J. Heek, K. Xiao, S. Agrawal, and J. Dean. Efficiently scaling transformer inference. *arXiv preprint arXiv:2211.05102*, 2022.

[27] J. W. Rae, A. Potapenko, S. M. Jayakumar, C. Hillier, and T. P. Lillicrap. Compressive transformers for long-range sequence modelling. In *International Conference on Learning Representations*, 2020.

[28] C. Raffel, N. Shazeer, A. Roberts, K. Lee, S. Narang, M. Matena, Y. Zhou, W. Li, and P. J. Liu. Exploring the limits of transfer learning with a unified Text-to-Text Transformer. *The Journal of Machine Learning Research*, 21(1):5485–5551, 2020.

[29] J. Rasley, S. Rajbhandari, O. Ruwase, and Y. He. DeepSpeed: System optimizations enable training deep learning models with over 100 billion parameters. In *Proceedings of the 26th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining*, pages 3505–3506, 2020.

[30] C. Snell, D. Klein, and R. Zhong. Learning by distilling context. *arXiv preprint arXiv:2209.15189*, 2022.

[31] R. Taori, I. Gulrajani, T. Zhang, Y. Dubois, X. Li, C. Guestrin, P. Liang, and T. B. Hashimoto. Stanford Alpaca: An instruction-following LLaMA model. `https://github.com/tatsu-lab/stanford_alpaca`, 2023.

[32] Y. Tay, M. Dehghani, D. Bahri, and D. Metzler. Efficient transformers: A survey. *ACM Computing Surveys*, 55(6):1–28, 2022.

[33] H. Touvron, T. Lavril, G. Izacard, X. Martinet, M.-A. Lachaux, T. Lacroix, B. Rozière, N. Goyal, E. Hambro, F. Azhar, et al. LLaMA: Open and efficient foundation language models. *arXiv preprint arXiv:2302.13971*, 2023.

[34] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, and I. Polosukhin. Attention is all you need. *Advances in Neural Information Processing Systems*, 30, 2017.

[35] J. Wang, Y. Liang, F. Meng, H. Shi, Z. Li, J. Xu, J. Qu, and J. Zhou. Is ChatGPT a good NLG evaluator? A preliminary study. *arXiv preprint arXiv:2303.04048*, 2023.

[36] Y. Wang, Y. Kordi, S. Mishra, A. Liu, N. A. Smith, D. Khashabi, and H. Hajishirzi. Self-Instruct: Aligning language model with self generated instructions. *arXiv preprint arXiv:2212.10560*, 2022.

[37] Y. Wang, S. Mishra, P. Alipoormolabashi, Y. Kordi, A. Mirzaei, A. Naik, A. Ashok, A. S. Dhanasekaran, A. Arunkumar, D. Stap, et al. Super-NaturalInstructions: Generalization via declarative instructions on 1600+ NLP tasks. In *Proceedings of the 2022 Conference on Empirical Methods in Natural Language Processing*, pages 5085–5109, 2022.

[38] J. Wei, M. Bosma, V. Zhao, K. Guu, A. W. Yu, B. Lester, N. Du, A. M. Dai, and Q. V. Le. Finetuned language models are zero-shot learners. In *International Conference on Learning Representations*, 2022.

[39] J. Wei, X. Wang, D. Schuurmans, M. Bosma, F. Xia, E. H. Chi, Q. V. Le, D. Zhou, et al. Chain-of-thought prompting elicits reasoning in large language models. In *Advances in Neural Information Processing Systems*, 2022.

[40] D. Wingate, M. Shoeybi, and T. Sorensen. Prompt compression and contrastive conditioning for controllability and toxicity reduction in language models. In *Findings of the Association for Computational Linguistics: EMNLP 2022*, pages 5621–5634, Abu Dhabi, United Arab Emirates, Dec. 2022. Association for Computational Linguistics. URL https://aclanthology.org/2022.findings-emnlp.412.

[41] T. Wolf, L. Debut, V. Sanh, J. Chaumond, C. Delangue, A. Moi, P. Cistac, T. Rault, R. Louf, M. Funtowicz, J. Davison, S. Shleifer, P. von Platen, C. Ma, Y. Jernite, J. Plu, C. Xu, T. Le Scao, S. Gugger, M. Drame, Q. Lhoest, and A. Rush. Transformers: State-of-the-art natural language processing. In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing: System Demonstrations*, pages 38–45, Online, Oct. 2020. Association for Computational Linguistics. doi: 10.18653/v1/2020.emnlp-demos.6. URL https://aclanthology.org/2020.emnlp-demos.6.

[42] Y. Wu, M. N. Rabe, D. Hutchins, and C. Szegedy. Memorizing transformers. In *International Conference on Learning Representations*, 2022.

[43] H. Zhang, Y. Gong, Y. Shen, W. Li, J. Lv, N. Duan, and W. Chen. Poolingformer: Long document modeling with pooling attention. In *International Conference on Machine Learning*, pages 12437–12446. PMLR, 2021.

[33] H. Touvron, T. Lavril, G. Izacard, X. Martinet, M.-A. Lachaux, T. Lacroix, B. Rozière, N. Goyal, E. Hambro, F. Azhar, et al. LLaMA: Open and efficient foundation language models. *arXiv preprint arXiv:2302.13971*, 2023.

[34] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, and I. Polosukhin. Attention is all you need. *Advances in Neural Information Processing Systems*, 30, 2017.

[35] J. Wang, Y. Liang, F. Meng, H. Shi, Z. Li, J. Xu, J. Qu, and J. Zhou. Is ChatGPT a good NLG evaluator? A preliminary study. *arXiv preprint arXiv:2303.04048*, 2023.

[36] Y. Wang, Y. Kordi, S. Mishra, A. Liu, N. A. Smith, D. Khashabi, and H. Hajishirzi. Self-Instruct: Aligning language model with self generated instructions. *arXiv preprint arXiv:2212.10560*, 2022.

[37] Y. Wang, S. Mishra, P. Alipoormolabashi, Y. Kordi, A. Mirzaei, A. Naik, A. Ashok, A. S. Dhanasekaran, A. Arunkumar, D. Stap, et al. Super-NaturalInstructions: Generalization via declarative instructions on 1600+ NLP tasks. In *Proceedings of the 2022 Conference on Empirical Methods in Natural Language Processing*, pages 5085–5109, 2022.

[38] J. Wei, M. Bosma, V. Zhao, K. Guu, A. W. Yu, B. Lester, N. Du, A. M. Dai, and Q. V. Le. Finetuned language models are zero-shot learners. In *International Conference on Learning Representations*, 2022.

[39] J. Wei, X. Wang, D. Schuurmans, M. Bosma, F. Xia, E. H. Chi, Q. V. Le, D. Zhou, et al. Chain-of-thought prompting elicits reasoning in large language models. In *Advances in Neural Information Processing Systems*, 2022.

[40] D. Wingate, M. Shoeybi, and T. Sorensen. Prompt compression and contrastive conditioning for controllability and toxicity reduction in language models. In *Findings of the Association for Computational Linguistics: EMNLP 2022*, pages 5621–5634, Abu Dhabi, United Arab Emirates, Dec. 2022. Association for Computational Linguistics. URL https://aclanthology.org/2022.findings-emnlp.412.

[41] T. Wolf, L. Debut, V. Sanh, J. Chaumond, C. Delangue, A. Moi, P. Cistac, T. Rault, R. Louf, M. Funtowicz, J. Davison, S. Shleifer, P. von Platen, C. Ma, Y. Jernite, J. Plu, C. Xu, T. Le Scao, S. Gugger, M. Drame, Q. Lhoest, and A. Rush. Transformers: State-of-the-art natural language processing. In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing: System Demonstrations*, pages 38–45, Online, Oct. 2020. Association for Computational Linguistics. doi: 10.18653/v1/2020.emnlp-demos.6. URL https://aclanthology.org/2020.emnlp-demos.6.

[42] Y. Wu, M. N. Rabe, D. Hutchins, and C. Szegedy. Memorizing transformers. In *International Conference on Learning Representations*, 2022.

[43] H. Zhang, Y. Gong, Y. Shen, W. Li, J. Lv, N. Duan, and W. Chen. Poolingformer: Long document modeling with pooling attention. In *International Conference on Machine Learning*, pages 12437–12446. PMLR, 2021.

A Example PyTorch Implementation of Gist Masking

See Listing A.1 for a sample annotated implementation of gist masking. This PyTorch implementation relies on basic NumPy-style tensor operations and can thus be adapted easily to a framework like JAX.

B Data, Training, Evaluation, and Compute Details

Code, data, and model checkpoints are available at `https://github.com/jayelm/gisting`.

Data. For LLaMA-7B, we used a maximum sequence length of 512 tokens during training and evaluation, except with the Human validation split, where the maximum length was increased to 768 (the Human instructions are longer). Examples longer than this length are truncated from the end. For FLAN-T5-XXL, we set a maximum input length (task t + input x) of 128 and a maximum output length of 256, except again for the Human split, where the maximum input and output lengths were both set to 384. For both models, we set a maximum generation length of 512 tokens. These lengths were chosen such that $< 1\%$ of examples across the board were truncated during training and evaluation for both models.

Training. Full hyperparameters for training runs are located in Table A.1. These parameters were adapted from previous published work finetuning LLAMA/FLAN-T5. For LLaMA-7B, parameters are identical to those used in training Alpaca Taori et al. [31]. For FLAN-T5-XXL, parameters are identical to those used in training Tk-INSTRUCT [37], except with a 5e-5 learning rate, as used in the Tk-INSTRUCT GitHub repository,[4] rather than the 1e-5 learning rate in the paper.

LLaMA-7B was trained for 3000 steps, while FLAN-T5-XXL was trained for 16000 steps. Since there are about 130k examples in Alpaca+, given the batch sizes in Table A.1 this corresponds to about ~3 epochs and ~2 epochs of training, respsectively. These numbers, again, are identical to Taori et al. [31] and Wang et al. [36]. We note that the training time is relatively flexible; for example, we did not see substantial gains training beyond 1 epoch for FLAN-T5-XXL.

Evaluation. During evaluation and benchmarking, we simply greedily decoded the most likely sequence. We saw limited gains from beam search with beam size $B = 4$.

Compute. Experiments were run on a cluster machine with 4xA100-SXM4-80GB NVIDIA GPUs, 480GB RAM, and 16 CPUs, using PyTorch 2.0 [24], Hugging Face Transformers [41], and DeepSpeed [29]. Training runs take about ~7 hours to complete for LLaMA-7B and ~25 hours for FLAN-T5-XXL. Benchmarking results were obtained on the same machine, but using just 1 of the A100 GPUs.

Table A.1: **Hyperparameters for training runs.**

	LLaMA-7B	FLAN-T5-XXL
num steps	3000	16000
num train epochs	≈ 3	≈ 2
batch size	128	16
learning rate	2e-5	5e-5
warmup ratio	0.03	0
precision	bf16	bf16
optimizer	AdamW	AdamW
Deepspeed		
# GPUs (A100 80GB)	4	4
ZeRO stage	3	3
subgroup size	1e9	1e9
max live params	1e9	1e9
max reuse distance	1e9	1e9

[4] `https://github.com/yizhongw/Tk-Instruct/blob/1ab6fad/scripts/train_tk_instruct.sh`

一个关于摘要单元掩码的ExamplePyTorch框架实现示例

有关摘要单元掩码的示例标注实现，可参见列表A.1。该PyTorch框架实现依赖于基础的NumPy风格张量运算，因此可以轻松适配到JAX之类的框架中。

B数据、训练、评估及计算细节

代码、数据以及模型检查点均可在此处获取：https://github.com/jayelm/gisting .

数据。对于LLaMA-7B模型，在训练和评估阶段我们设定的最大序列长度为512个标记，但人类验证数据集除外，该数据集的最大长度被提升至768（因为人类指令集的长度更长）。超过此长度的示例会从末尾开始被截断。而对于FLAN-T5-XXL模型，我们设置的最大输入长度（任务 t +输入 x）为128，最大输出长度为256，同样人类验证数据集除外，该数据集的输入和输出最大长度均设为384。针对这两种模型，我们都将最大生成长度设定为512个标记。选择这些长度是为了确保在训练和评估过程中，两种模型各有 $< 1\%$的示例会被截断。

训练。训练过程所使用的全部超参数详见表A.1。这些参数是根据此前已发表的针对LLAMA/FLAN-T5的微调研究结果调整而来的。对于LLaMA-7B，其参数与Taori等人 [31]在训练Alpaca模型时所使用的参数完全相同。至于FLAN-T5-XXL，其参数也与Tk-INSTRUCT GitHub仓库[4] 中采用的参数一致，即学习率为5e-5，而非该论文中提到的1e-5，只不过后者用于训练Tk-INSTRUCT [37], 模型。

LLaMA-7B模型的训练步数为3000步，而FLAN-T5-XXL模型的训练步数为16000步。由于Alpaca+中包含约13万条样本，结合表A.1中的批量大小，这分别对应大约 ~3 个训练周期和 ~2 个训练周期。这些数值与Taori等人的 [31] 以及Wang等人的 [36]研究结果完全一致。需要指出的是，训练时间具有较大的灵活性；例如，对于FLAN-T5-XXL模型，我们在超过1个训练周期后并未观察到明显的性能提升。

评估。在评估和基准测试过程中，我们只是简单地以贪心方式解码出现概率最高的序列。使用波束宽度为 $B = 4$的波束搜索方法，并未带来明显的提升效果。

计算部分。实验是在一台集群机器上进行的，该机器配备了4块NVIDIA A100-SXM4-80GB图形处理器、480GB内存以及16个CPU，同时使用了PyTorch 2.0 [24], 、Hugging Face Transformers [41], 和DeepSpeed[29]。对于LLaMA-7B模型，训练过程大约需要 ~7 小时才能完成；而对于FLAN-T5-XXL模型，则需要 ~25 小时。基准测试结果也是在同一台机器上获得的，只不过仅使用了其中1块A100图形处理器。

表A.1：**训练实验所使用的超参数**。

	LLaMA-7B模型	FLAN-T5-XXL模型
训练步数	3000	16000
训练轮数	≈ 3	≈ 2
批量大小	128	16
学习率	2e-5	5e-5
热身比例	0.03	0
精确度	bf16	bf16
优化器	AdamW算法	AdamW算法
DeepSpeed框架		
# 图形处理器（A100 80GB）	4	4
ZeRO阶段	3	3
子组大小	1e9	1e9
最大活跃参数数量	1e9	1e9
最大重用距离	1e9	1e9

[4] https://github.com/yizhongw/Tk-Instruct/blob/1ab6fad/scripts/train_tk_instruct.sh

```python
import torch


def reverse_cumsum(x: torch.Tensor) -> torch.Tensor:
    """Cumulative sum from right to left.

    See https://github.com/pytorch/pytorch/issues/33520.
    """
    return x + torch.sum(x, dim=-1, keepdims=True) - torch.cumsum(x, dim=-1)


def make_mask_pre_first_gist(inputs: torch.Tensor, gist_token: int, dtype=torch.int64) -> torch.Tensor:
    """Returns a mask where all tokens prior to the first gist token are masked out.

    Args:
        inputs: a Tensor of input tokens where the last dimension is the sequence length.
        gist_token: the integer id of the gist token.
        dtype: the dtype of the mask, default int64.
    Returns:
        The requested mask.
    """
    return ((inputs == gist_token).cumsum(-1) >= 1).type(dtype)


def make_mask_post_last_gist(inputs: torch.Tensor, gist_token: int, dtype=torch.int64) -> torch.Tensor:
    """Returns a mask where all tokens after the last gist token are masked out.

    Computes the same as mask_pre_first_gist_token, but reverses the sequence before and after the cumsum.

    Args:
        inputs: a Tensor of input tokens where the last dimension is the sequence length.
        gist_token: the integer id of the gist token.
        dtype: the dtype of the mask, default int64.
    Returns:
        The requested mask.
    """
    return (reverse_cumsum(inputs == gist_token) >= 1).type(dtype)


def make_gist_mask(inputs: torch.Tensor, gist_token: int, pad_token: int, dtype=torch.int64) -> torch.Tensor:
    """Creates a gist mask from supplied inputs and gist/pad tokens.

    Tokens after the last gist cannot attend to tokens prior to the first gist. Additionally, tokens *before*
    the last gist cannot attend to tokens *after* the last gist.

    The gist mask is broadcasted to 4D (with a singleton dim 1) for compatibility with multi-headed attention
    (where dim 1 is the head dimension).

    Args:
        inputs: a Tensor of shape (batch_size, seq_len) input tokens.
        gist_token: the integer id of the gist token.
        pad_token: the integer id of the pad token. inputs == pad_token are masked out.
        dtype: the dtype of the mask, default int64.
    Returns:
        The requested mask of shape (batch_size, 1, seq_len, seq_len)
    """
    # Attention mask for tokens before the last gist token.
    pre_gist_mask = make_mask_post_last_gist(inputs, gist_token, dtype=torch.bool)[:, None, None]
    # Attention mask for tokens after the last gist token.
    post_gist_mask = make_mask_pre_first_gist(inputs, gist_token, dtype=torch.bool)[:, None, None]
    # Construct time masks by permuting to time dimension.
    pre_gist_time_mask = pre_gist_mask.permute((0, 1, 3, 2))

    mask = torch.where(pre_gist_time_mask, pre_gist_mask, post_gist_mask)
    mask = mask & (inputs != pad_token)[:, None, None]  # Mask out pad tokens.

    return mask.type(dtype)
```

Listing A.1: **Sample PyTorch implementation of gist masking.**


```python
import torch


def reverse_cumsum(x: torch.Tensor) -> torch.Tensor:
    从右到左的累加和。

    请参阅 https://github.com/pytorch/pytorch/issues/33520。
    """
    返回x + torch。sum(x, 暗淡=-1, keepdims=True) - torch.cumsum(x, 暗淡=-1)


def make_mask_pre_first_要点摘要(输入标记张量: torch.Tensor, 要点摘要_token:int, 数据类型=torch。int64) -> torch。Tensor:
    返回一个掩码，其中第一个要点摘要标记之前的所有标记都会被遮蔽。

    参数:
        输入标记张量：即由输入标记构成的张量，其最后一维代表序列长度。
        要点摘要_标记的整数值标识，用于表示对应的要点摘要标记。
        数据类型：指该掩码所采用的数据类型。 默认值为int64。
    返回值:
        用户请求的对应掩码。
    """
    返回((输入标记张量 == 要点摘要_令牌)。累积求和操作(-1) >= 1)。数据类型


def make_mask_post_last_gist(inputs: torch.Tensor, gist_token: int, 数据类型=torch。int64) -> torch。Tensor:
    返回一个掩码，该掩码会将最后一个要点摘要标记之后的所有标记都遮蔽掉。

    计算方式与mask_pre_first_gist_token相同。 但会反转累积求和和操作前后的顺序。

    参数:
        inputs：一个包含输入标记张量的数据结构，其最后一个维度表示序列长度。
        gist_token：要点摘要标记的整数编号。
        dtype：掩码的数据类型。 默认值为 int64。
    返回值:
        所需的掩码。
    """
    return(reverse_cumsum(inputs == 要点摘要_token) >= 1).type(数据类型)


def make_要点摘要_掩码（输入标记张量: torch。Tensor, 要点摘要_token: int, 填充_token: int, 数据类型=torch。int64) -> torch。Tensor:
    根据所提供的输入标记张量以及要点摘要/填充令牌，生成摘要单元掩码。

    最后一个要点摘要之后的提示词无法关联到第一个要点摘要之前的提示词。此外， 在之前的那些标记之后。
    最后一个要点摘要无法关注到该最后一个要点摘要之后的标记。

    为与多头注意力机制兼容，要点摘要单元掩码会被广播为4维结构（其中包含一个值为1的单一维度）。
    这里的维度1即表示注意力头维度。

    参数:
        inputs：形状为(batch_size, seq_len)的输入标记张量。
        gist_token：要点摘要标记的整数编号。
        pad_token：填充标记的整数编号；inputs ==pad_token这些内容会被屏蔽掉。
        dtype：用于表示该屏蔽值的数据类型。默认值为int64。
    返回值:
        形状为(batch_size)的所需屏蔽矩阵。 1，seq_len，seq_len)
    """
    # 用于最后一个要点摘要标记之前的标记的注意力掩码。
    pre_要点摘要_掩码 = 生成_掩码_输出_最后_要点摘要(输入标记张量, 要点摘要_提示词, 数据类型=torch。bool)[、 无、 None]
    # 最后一个要点摘要标记之后的标记注意力掩码。
    正面评价_要点摘要_掩码 = 生成_掩码_前置_首个_要点摘要(输入标记张量, 要点摘要_提示词, 数据类型=torch。bool)[、 无、 无]
    # 通过转置到时间维度来构建时间掩码。
    pre_要点摘要_时间_掩码 =pre_要点摘要_掩码。permute((0, 1, 3, 2))

    掩码 =torch.where(pre_正面评价_时间_掩码, pre_正面评价_掩码, post_正面评价_掩码)
    遮罩 =遮罩 &(输入标记张量 !=填充_标记)[:,无、 无] # 隐藏填充提示词。

    返回掩码。类型（数据类型）
```

表A.1：**要点摘要掩码功能的PyTorch实现示例。**

C Details of ChatGPT Evaluation

```
Given a user instruction and two AI assistant responses, your job is to select the response that is
more helpful, and give reasons why. Judge responses holistically, paying special attention to whether
the response (1) correctly follows the provided instruction and (2) is factually accurate. If both
responses are equal, you may call a tie, but please use this response sparingly.

Example 1:
```
Instruction: Given the name of a city, tell me the name of the state it is in.
Input: Los Angeles
Assistant A: California
Assistant B: Wyoming
```

Your output should be:
```
{"reason": "Los Angeles is in California. Only Assistant A is correct.", "choice": "A"}
```

Example 2:
```
Instruction: Give me some examples of fun things to do with the kids on a weekend.
Assistant A: For example, what should I do with my kids on a weekend? What if I'm in a city? What if
I'm on the beach?
Assistant B: You could try going to the zoo or a museum. If you want to stay at home, you could play
board games or bake cookies.
```

Your output should be:
```
{"reason": "Assistant A doesn"t seem to be following the instruction. Assistant B provides helpful
information with examples.", "choice": "B"}
```

Example 3:
```
Instruction: Write me a python function that prints "Hello, World".
Assistant A: def hello_world(): print("Hello!")
Assistant B: "Hello, World"
```

Your output should be:
```
{"reason": "Assistant B just wrote 'Hello, World'. Assistant A actually wrote a Python function, even
if it doesn't exactly print the right thing, so overall Assistant A is better.", "choice": "A"}
```

Example 4:
```
Instruction: Translate the following sentence from English to French.
Input: I like cats.
Assistant A: Me gustan los gatos.
Assistant B: 我喜欢猫.
```

Your output should be:
```
{"reason": "Both assistants got the language wrong.", "choice": "tie"}
```

Your response should only be in the JSON format above; THERE SHOULD BE NO OTHER CONTENT INCLUDED IN
YOUR RESPONSE. Write the "reason" key before writing the "choice" key, so that you think step-by-step
before making your decision. KEEP YOUR REASONING BRIEF.
```

Listing A.2: **Full prompt given to ChatGPT for evaluation**. This prompt populates the `system` field in the ChatGPT API; the actual example to be evaluated is formatted like the examples in the prompt above, then given as the sole input in the `user` field.

We used the ChatGPT API, specifically the `chatgpt-3.5-turbo` engine, to run our ChatGPT evaluation experiments over a period of 2 weeks between March 27 and April 7, 2023.

The full prompt given to ChatGPT is located in Listing A.2, and contains 4 examples of desired output from ChatGPT, including preferring factually accurate responses (Example 1), preferring responses that follow the instruction, even if imperfect (Examples 2 and 3), and examples of models

C ChatGPT评估详情

```
给定一个用户指令以及两个人工智能助手的回复，你的任务是选出更实用的回复，并说明理由。需对回复进行全面
评估，尤其要关注其是否（1）准确遵循给定的指令，以及（2）在事实层面是否正确。如果两个回复表现相当，你
可以判定为平局，但请尽量少使用这一选项。

示例1：、、、

指令：给出某座城市的名称，                然后告诉我它所在的州名称。
输入内容：洛杉矶
助手A：加利福尼亚州
助手B：怀俄明州
```

您的输出应为：
```
"原因":    "洛杉矶位于加利福尼亚州，因此只有助手A的回答是正确的。"        "选择结果": "A"}
```

示例2：
```
指令：给我一些周末可以和孩子一起做的有趣事情的例子。
助手A：比如，周末我该带孩子们做些什么呢？如果我在城市里又会怎样？要是……又该如何处理？
我人在海滩上吗？
助手B：你可以试试去动物园或博物馆。如果你想待在家里，也可以玩一玩。
玩棋盘游戏或烤饼干。
```

您的输出应为：
```
{"reason": "把无相关解释内容"}    助手A似乎没有按照指示行事。而助手B则给出了很有用的帮助。
附带示例的说明信息。          ，"选择项"  "B"}
```

示例3：
```
指令：为我编写一个能打印输出的Python函数              您好，    “世界”。
助手A：def hello_world(): print("Hello!")
助手B：        您好  ，世界
```

您的输出应为：
```
{"reason": "把无相关解释内容"}    助手B刚刚写了          您好，    “世界”。助手A甚至还编写了一个Python函数。
如果输出的结果并非完全正确的话，          那么总体而言，助手A的表现更好。      "choice": "A"}
```

示例4：
```
任务要求：将以下句子从英语翻译成法语。
输入内容：我喜欢猫。
助手A：我喜欢猫。
助手B：我喜欢猫。
```

您的输出应为：
```
"原因":    "两位助手都用错了语言。",                "选择":    {"tie":"平
                                                                局"}
```

您的回复必须仅采用上述JSON格式；不得包含任何其他内容。
请给出您的回复。在填写“选择”键之前的内容中，请先写上“原因”键，这样有助于您逐步思考。
在做出决定之前，请保持思路简洁明了。

列表A.2：**提交给ChatGPT用于评估的完整提示词**。该提示词会填充ChatGPT API中的system字段；实际需要评估的示例会按照上述提示词中的格式进行编排，随后作为唯一输入内容填入user字段中。

在2023年3月27日至4月7日的2周时间里，我们使用了ChatGPT API，具体来说是chatgpt-3.5-turbo引擎，来开展ChatGPT评估实验。

提供给ChatGPT的完整提示词见列表A.2，其中包含了4个期望从ChatGPT处获得的输出示例：优先选择事实准确的回复（示例1），即便不够完美也优先选择遵循指令的回复（示例2和3），以及模型输出完全错误的情况（示例4）。在对比的这两款模型中，我们随机决定了每种模型的呈现顺序，将其表现为助手A或助手B，以此避免顺序带来的影响。

Table A.2: **Pairwise Cohen's κ** between human annotators (H1, H2, H3) and ChatGPT.

(a) LLaMA-7B

	H1	H2	H3	**Average**
H1	–	.21	.33	.27
H2	.21	–	.19	.20
H3	.33	.19	–	.26
ChatGPT	.38	.22	.26	.29

(b) FLAN-T5-XXL

	H1	H2	H3	**Average**
H1	–	.33	.34	.34
H2	.33	–	.33	.33
H3	.34	.33	–	.33
ChatGPT	.35	.18	.34	.29

being equally wrong (Examples 4). For the two models under comparison, we randomized the order of presentation of each model as either Assistant A or Assistant B, to avoid order effects.

ChatGPT was instructed to only respond in JSON format, outputting first a `reason` key followed by a `choice` key, to encourage chain-of-thought reasoning [39]. On rare occasions ($< 0.25\%$ of the time), ChatGPT would output a response that did not conform to the requested JSON format (e.g. it would just give an unstructured paragraph). In these cases we manually went through and converted these responses to JSON, without altering ChatGPT's reasoning.

In total, we collected ~22.5k judgments from ChatGPT for an estimated cost of $29.28. The full outputs for each model across the Alpaca+ validation splits, as well as ChatGPT's responses and choices, are available in the code link above.

D Additional Human Evaluation Details and Results

D.1 Experimental Details

For each of the 100 examples randomly selected from the Human validation split, we recruited 3 US or UK-based, English-fluent annotators from Prolific, an online crowdsourcing platform. Experiments were IRB approved under a generic human experiments IRB given to the authors.

The annotation interface given to Prolific crowdworkers is located in Figure A.1. To verify task comprehension, participants were shown two simple examples before the main body of the task (Figure A.2), and were required to answer correctly before proceeding. We compensated participants USD $14.35/hour for an estimated cost (including Prolific fees) of USD $141.64.

D.2 Additional Results

See Table A.2 for a breakdown of Cohen's κ between human annotators and ChatGPT. We used a weighted version of Cohen's κ with linear weights, since the response scale is ordinal (e.g. "tie" is a closer judgment to "pos control win" than "gist win").

E Exact Match Results

See Figure A.3 for a plot of exact match rates for the gist and positive control models (as measured by exact string match).

F Additional FLOPs details

The FLOPs required for a Transformer formward pass with varying KV cache lengths can be estimated by modifying existing equations to account for self-attention back to the KV cache. As an example, we modify the FLOPs equations used for computing FLOPs in the Chinchilla paper (Appendix F in [14]). Let **seq_len_with_past =** seq_len + kv_cache_len. Then the modified Transformer FLOPs equations are:

Embeddings

- $2 \times$ seq_len $\times$ vocab_size $\times$ d_model

表A.2：**人类注释者（H1、H2、H3）与ChatGPT之间的成对科恩系数 κ**

(a) LLaMA-7B

	H1	H2	H3	**平均值**
H1	–	.21	.33	.27
H2	.21	–	.19	.20
H3	.33	.19	–	.26
ChatGPT	.38	.22	.26	.29

(b) FLAN-T5-XXL

	H1	H2	H3	**平均值**
H1	–	.33	.34	.34
H2	.33	–	.33	.33
H3	.34	.33	–	.33
ChatGPT	.35	.18	.34	.29

对于正在被比较的这两款模型，我们还会随机安排它们作为助手A还是助手B来展示，如此做是为了排除顺序因素可能带来的偏差。

我们要求ChatGPT仅以JSON格式作答，先输出原因键，再输出选项键，以此来促进其进行链式思维推理 [39]。在极少数情况下（占比$< 0.25\%$），ChatGPT会输出不符合要求的JSON格式的回应（例如直接给出一段无结构的文字）。对于这类情况，我们会手动将其转换成JSON格式，同时不会改动ChatGPT的推理过程。

总共，我们从ChatGPT处收集了 ~22.5k条评估结果，预估花费为29.28美元。上述代码链接中提供了Alpaca+ 验证分组下各模型的完整输出，以及ChatGPT的响应和选择情况。

D 其他人工评估的详细信息与结果

D.1 实验细节

对于从人类验证数据集中随机选取的100个示例，我们分别从在线众包平台Prolific中招募了3名来自美国或英国、英语流利的标注员。这些实验已获得机构审查委员会的批准，审批依据是授予作者的通用人类实验机构审查委员会许可。

提供给Prolific众包工作者的标注界面如图A.1所示。为验证他们对任务的理解程度，参与者在开始正式任务之前会先看到两个简单示例（见图A.2），并且必须正确回答这些问题才能继续。根据约141.64美元的预估成本（包括Prolific的手续费），我们向参与者支付了每小时14.35美元的报酬。

D.2 其他结果

如需了解人类注释者与ChatGPT之间的Cohen's κ 具体数值，可参见表A.2。由于响应等级属于有序类型（例如，“平局”的评分距离“正面评价控制组获胜”比距离“要点摘要获胜”更近），我们采用了带线性权重的Cohen's κ 加权版本来进行计算。

E 精确匹配结果

请参见图A.3，其中展示了要点摘要模型与正控制模型的精确匹配率曲线（通过完全字符串匹配进行衡量）。

F 额外浮点运算次数详情

通过修改现有公式并纳入指向KV缓存的自注意力机制，即可估算出具有不同KV缓存长度的Transformer模型前向传播过程所需的浮点运算次数。例如，我们参考了[14]中Chinchilla论文（附录F）中用于计算浮点运算次数的公式，并将其修改为**seq_len_with_past** =seq_len + kv_cache_len的形式。由此得到的修改后的Transformer模型浮点运算次数公式如下：

嵌入向量

- $2 \times$ 序列长度为_，词汇表大小为 $\times$ ，参数维度为_，所使用的模型为 $\times$ d_结构。

Judging AI Assistants

Progress: 12/50

User Instruction: Write a good Twitter Bio. Try to cover all the provided main points in your generated bio.

User Input: Name: Jane Main points: - Architect - Interested in Tech Written in first person Use casual tone

---

Assistant A: Jane is an architect who is interested in tech. She loves to solve problems and create beautiful things.

Assistant B: I'm Jane, an architect who loves to explore the latest tech trends. I'm always looking for new ways to make our lives easier and more convenient. Follow me for the latest tech news, tips, and inspiration!

Which Assistant is more helpful?

A | B | TIE | I DON'T KNOW

Remember:

When making your judgment, please consider whether the response
1. attempts to follow the provided instruction, and
2. is factually accurate.

If you can't decide, e.g. because the responses are the same or equally good or bad, you may call a tie, but please use this response sparingly.

Finally, if you do not understand the question being asked: you may still be able to make
an educated guess about which Assistant is better, for example if one assistant is clearly
not following the directions. If so, please do! Otherwise, click the
"I don't know" button. There is no need to look up additional information or
spend a huge amount of time on an example; just click "I don't know.".

Figure A.1: **Annotation interface given to Prolific crowdworkers.**

User Instruction: Give me some examples of fun things to do with the kids on a weekend.

Assistant A: For example, what should **I** do with my kids on a weekend? What if **I**'m in a city? What if **I**'m on the beach?

Assistant B: You could try going to the zoo or a museum. If you want to stay at home, you could play board games or paint photographs.

User Instruction: Find me a synonym of the following word.

User Input: speedy

Assistant A: quick

Assistant B: lazy

Figure A.2: **Example items given to humans before the start of the task.**

评测人工智能助手

进度：12/50

用户指令：撰写一段优秀的 Twitter 个人简介。请尽力在生成的简介中涵盖所有提供的要点。

用户输入：姓名：简 主要特点：- 架构师 - 对科技类领域感兴趣 以第一人称撰写，采用轻松的语气

助手A：简是一名对科技类领域感兴趣的建筑师，她非常热衷于解决问题。解决问题并创造美好的事物。

助手B：我是简，一名热衷于探索最新科技趋势的建筑师。我一直在寻找能让我们的生活更加轻松便捷的新方法。关注我，获取最新的科技资讯、实用技巧以及灵感启迪！

哪款助手更实用？

A | B | TIE | 我不知道。

记住：

在作出判断时，请考虑该响应是否：1. 尝试遵循给定的指示，以及 2. 在事实层面准确无误。

如果无法做出决定，例如因为各条回复内容相同，或者优劣程度相当，可以将其视为平局，但请尽量少使用这种判定方式。

最后，如果您不明白所提出的问题：您或许仍然能够给出答案。

针对哪款助手表现更好做一个合理的推测，比如当某款助手明显没有按照指示操作时。如果是这种情况，请这样做！否则，请点击“我不知道”按钮。无需查找额外信息，也不用在某个示例上花费大量时间，直接点击“我不知道”即可。

图A.1：**提供给Prolific众包工作者的标注界面**。

User Instruction: Give me some examples of fun things to do with the kids on a weekend.

Assistant A: For example, what should **I** do with my kids on a weekend? What if **I**'m in a city? What if **I**'m on the beach?

Assistant B: You could try going to the zoo or a museum. If you want to stay at home, you could play board games or paint photographs.

User Instruction: Find me a synonym of the following word.

User Input: speedy

Assistant A: quick

Assistant B: lazy

图A.2：**任务开始前展示给人类参与者的示例内容**。

Attention (Single Layer)

- *Key, query, and value projections*: 2 × 3 × seq_len × d_model × (key_size × num_heads)
- *Key and query logits*: 2 × seq_len × **seq_len_with_past** × (key_size × num_heads)
- *Softmax*: 3 × num_heads × seq_len × **seq_len_with_past**
- *Softmax @ query reductions*: 2 × seq_len × **seq_len_with_past** × (key_size × num_heads)
- *Final linear*: 2 × seq_len × (key_size × num_heads) × d_model

Dense Block

- 2 × seq_len × (d_model × ffw_size + d_model × ffw_size)

Final Logits

- 2 × seq_len × d_model × vocab_size

Total Forward Pass FLOPs

- embeddings + num_layers × (attention_single_layer + dense_block) + final_logits

It can be seen that only 3 operations in each attention layer depend on the KV cache size, and they take up a relatively insignificant amount of FLOPs. As an illustrative example, Figure A.4 shows the relative FLOPs contributions within a single layer of attention for LLaMA-7B, assuming a *2000-length* KV cache and a single input token. Operations dependent on the KV cache constitute at most ~10% of the total attention layer FLOPs; the rest are used in KQV projections and dense layers for processing the single new input token.

Given a KV cache compression rate of 26, as observed in our Human validation split, the Chinchilla equations predict a relative improvement of Gist caching of 0.12%. This is extremely close to the 0.11% improvement actually observed in Table 3. These results show that optimizing the KV cache size does not actually lead to huge compute speedups during Transformer inference, at least for relatively small prompt lengths. Nevertheless, there are clear memory and storage benefits to be gained from prompt compression, as discussed in Section 6.

G Additional Examples of Model Outputs

See Listing A.3 for additional examples of instructions, inputs, model completions (for LLaMA-7B and FLAN-T5-XXL across Pos, Gist, TF-IDF, and Neg conditions), human judgments, and ChatGPT judgments (+reasons). Examples are not cherry-picked; we enumerate the first 10 examples of the Alpaca+ Human validation split.

单层注意力机制

- 键、查询项以及值投影： 2 × 3 × 序列_长度 × d_模型 × (键的_大小 × 数量_头数)
- 键与查询对数几率： 2 × seq_len × **seq_len_与_之前的** × （键_大小 × 数量_头数）
- *softmax*函数： 3 × 个_输出头， × 序列_长度为 × **序列_长度_结合_历史信息**
- 基于查询的*softmax*简化结果： 2 × seq_len × **seq_len_与_之前的** × (key_size × num_heads)
- 最终线性层： 2 × seq_len × (key_size × num_heads) × d_模型

密集块

- 2 × seq_len × (d_模型 × 前向传播宽度 _大小 + d_模型 × 前向传播宽度 _大小)

最终对数几率层

- 2 × seq_len × d_模型 × 词汇表_大小

整个前向传播过程的浮点运算次数

- 嵌入向量 +数量_层 × （注意力_单_层 + 密集_块） + 最终_逻辑值

可以看出，每个注意力层中仅有3项操作依赖于KV缓存的大小，而这些操作所消耗的浮点运算次数相对较少。作为示例，图A.4展示了在假设拥有<span style id='1'>长度为2000的KV缓存以及一个输入标记的情况下，LLaMA-7B模型单个注意力层中各部分对浮点运算次数的贡献比例。依赖KV缓存的操作最多仅占整个注意力层浮点运算次数的 ~10%，其余部分则用于KV投影以及处理该单个新输入标记的密集层运算中。
根据我们的人类验证数据集所观察到的26%的KV缓存压缩率，Chinchilla方程组预测要点缓存的相对提升幅度为0.12%。这一数值与表3中实际观测到的0.11%的提升幅度极为接近。这些结果表明，至少对于提示词长度相对较短的情况而言，优化KV缓存大小并不会在变换器推理过程中带来极大的计算速度提升。不过正如第6节所讨论的，通过提示词压缩仍然能够显著节省内存和存储资源。

G 模型输出的更多示例

更多关于指令、输入标记张量、模型输出结果（涵盖LLaMA-7B与FLAN-T5-XXL在正面评价、要点摘要、TF-IDF及负面评价条件下的表现）、人类评分以及ChatGPT评分（+原因）的示例，请参见列表A.3。这些示例并非经过刻意挑选，而是从Alpaca+人类验证数据集中选取的前10个案例。

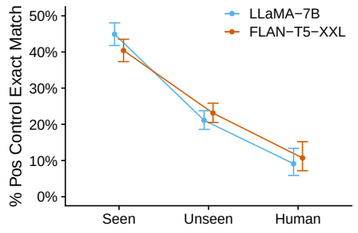


Figure A.3: **Exact match rates**. Rate at which the 1 token gist models give the same output as the positive control (exact string match). Error bars are 95% exact binomial confidence intervals.

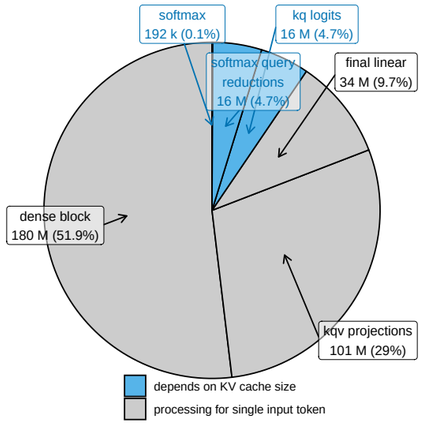
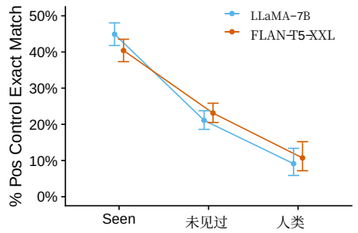
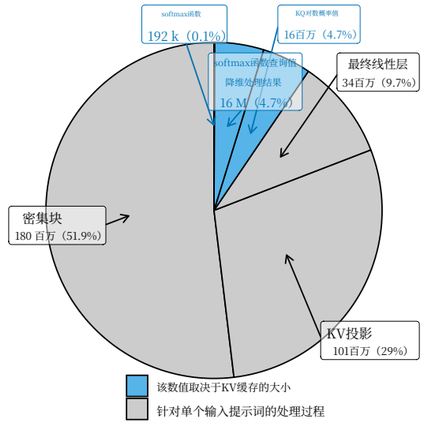


Figure A.4: **FLOPs for each operation involved in a single layer of self attention with a 2000-length KV cache**, according to the Chinchilla estimates for LLaMA-7B. At most 9.6% of FLOPs can be optimized away by reducing the size of the KV cache.



图A.3：**精确匹配率**。此处显示的是1个标记的概要模型给出与阳性对照组完全相同的输出（即完全字符串匹配）时的比例，而误差条则表示95%精确二项式置信区间。



图A.4：**具有2000长度KV缓存的单层自注意力机制中各项操作对应的浮点运算次数**，数据依据的是针对LLaMA-7B的Chinchilla估算值；通过缩小KV缓存的大小，最多可减少9.6%的浮点运算次数。

Example 1

Instruction: Write a good Twitter Bio. Try to cover all the provided main points in your generated bio. <G1>
Input: Name: Jane
Main points:
- Architect
- Interested in Tech
Written in first person
Use casual tone
Reference Output: I like to think of myself as a kind of architect.
I'm interested in the tech side of things, and I love coming up with new ideas for how to make things better.
TF-IDF Word: bio

LLaMA-7B

Pos: I'm Jane, an architect and tech enthusiast.
I'm interested in the latest trends and always looking to learn more.
I'm here to share my thoughts and experiences with the world.
Gist: I'm Jane, an architect and tech enthusiast.
I love to explore new places and learn about different cultures.
I'm always looking for fun things to do and new experiences.
I'm an avid reader and enjoy learning about the world around me.
I'm also a big fan of tech and love to stay up to date on the latest trends.
I'm looking forward to seeing what the world has to offer!
TF-IDF: Jane is an architect who is interested in technology.
She likes to stay up to date on the latest trends and is always looking for new ways to apply them in her work.
Jane is a creative problem solver who approaches each project with an innovative eye.
Neg: Jane is an architect who is interested in technology.
She likes to stay up to date on the latest trends and is always looking for new ways to incorporate them into her work.
She's an avid reader of tech news and loves to explore the latest gadgets.
Jane is also a big fan of the arts and enjoys visiting museums and galleries when she gets the chance.
Human choice (Pos vs Gist): Pos Tie **Gist**
ChatGPT choice (Pos vs Gist): Pos
ChatGPT reason (Pos vs Gist): Assistant A covers all the main points provided in the instruction, while Assistant B goes off-topic and doesn't mention the interest in tech as much. Assistant A also has a more professional tone, which is more appropriate for a Twitter bio. (Assistant A: **Pos**, Assistant B: **Gist**)

FLAN-T5-XXL

Pos: I'm Jane, an architect who loves to explore the latest tech trends.
I'm always looking for new ways to make our lives easier and more convenient.
Follow me for the latest tech news, tips, and inspiration!
Gist: Jane is an architect who is interested in tech.
She loves to solve problems and create beautiful things.
TF-IDF: Architect, Tech enthusiast
Neg: Jane is an architect who is interested in tech.
Human choice (Pos vs Gist): Pos Pos Pos
ChatGPT choice (Pos vs Gist): Pos
ChatGPT reason (Pos vs Gist): Assistant A covers all the main points and uses a casual tone. Assistant B is too brief and doesn't provide any specific information about Jane's interests or personality. (Assistant A: **Pos**, Assistant B: **Gist**)

Example 2

Instruction: Name the top cities in France that should not be missed. Include the best aspects of each place as well. <G1>
Reference Output: 1- Paris, Best for: Historic sights and romantic vibes
2- Nice, Best for: Architecture and dining
3- Lyon, Best for: Dining and live music
4- Bordeaux, Best for: Wine and antiques
5- Marseille, Best for: Museums and nightlife
6- Strasbourg, Best for: Historic architecture and dining
TF-IDF Word: missed

LLaMA-7B

Pos: The top cities in France that should not be missed include Paris, which is known for its iconic architecture and bustling nightlife, and Nice, which is a beautiful city on the Mediterranean known for its colorful culture and delicious cuisine.
Gist: The top cities in France that should not be missed are Paris, Nice, Lyon, Toulouse, Marseille, and Cannes.
The must-see attractions in each city include the Eiffel Tower in Paris, the Promenade des Anglais in Nice, the Old Town in Lyon, the Canal du Midi in Toulouse, the Port of Marseille in Marseille, and the Croisette in Cannes.
TF-IDF: I missed you.
Neg: The most popular social media platform in Europe is the Android-based Ovi from Nokia.
Human choice (Pos vs Gist): Pos Tie **Gist**
ChatGPT choice (Pos vs Gist): **Gist**
ChatGPT reason (Pos vs Gist): Assistant A provides a more comprehensive list of top cities in France and includes the best aspects of each place. Assistant B only mentions two cities and their general characteristics without providing specific attractions or details. (Assistant A: **Gist**, Assistant B: **Pos**)

FLAN-T5-XXL

Pos: The top cities in France that should not be missed include Paris, Marseille, Lyon, Toulouse, Nice, and Monaco.
Paris is the capital of France and is home to the Eiffel Tower, Notre-Dame de Paris, and the Louvre Museum.
Marseille is a beautiful city with a vibrant nightlife, and Lyon is a great place to visit for its food and wine.
Toulouse is a charming city with a vibrant nightlife, and Monaco is a beautiful place to visit for its casinos and luxurious hotels.
Gist: Paris is one of the most beautiful cities in France.
It is a great place to visit for its architecture, culture, and history.
The city is also known for its beautiful gardens and parks, as well as its many museums and art galleries.
The Eiffel Tower is one of the most iconic landmarks in the world, and it is a must-see for anyone visiting Paris.
The Louvre is another must-see for anyone visiting Paris.
It is a stunning museum that houses some of the most iconic art pieces in the world.
The Notre-Dame de Paris is another must-see for anyone visiting Paris.
TF-IDF: I would be very disappointed.
Neg: - The Shawshank Redemption - Forrest Gump - The Godfather - Pulp Fiction - The Dark Knight
Human choice (Pos vs Gist): Pos Pos Pos
ChatGPT choice (Pos vs Gist): **Gist**

示例 1

指令on: 请撰写一段出色的 Twitter 个人简介。在生成的简介中尽量涵盖所有给定的要点。 <G1>输入内容: 姓名：简
要点包括：- 建筑师身份 - 对科技类内容感兴趣 - 以第一人称撰写 - 采用轻松随意的语气
参考信息 Output: 我 lik𝑀e到t hm的墨水y将自己视作一种建筑师。
我'我对科技领域很感兴趣，也喜欢不断思考如何让事物变得更好。TF-IDF词条：个人简介

LLaMA-7B模型

正面评价: 'm Jane, an architect and tech enthusiast.
我。
I'm interested in the latest trends and always looking to learn more.
I'm here to share my thoughts and experiences with the world.
Gist: I'm Jane, an architect and tech enthusiast.
I love to explore new places and learn about different cultures.
I'm always looking for fun things to do and new experiences.
I'm an avid reader and enjoy learning about the world around me.
I'm also a big fan of tech and love to stay up to date on the latest trends.
I'm looking forward to seeing what the world has to offer!
TF-IDF: Jane is an architect who is interested in technology.
She likes to stay up to date on the latest trends and is always looking for new ways to apply them in her work.
Jane is a creative problem solver who approaches each project with an innovative eye.
Neg: Jane is an architect who is interested in technology.
She likes to stay up to date on the latest trends and is always looking for new ways to incorporate them into her work.
She's an avid reader of tech news and loves to explore the latest gadgets.
Jane is also a big fan of the arts and enjoys visiting museums and galleries when she gets the chance.
Human choice (Pos vs Gist): Pos Tie **Gist**
ChatGPT choice (Pos vs Gist): Pos
ChatGPT的回答是（正面评价 与要点摘要）： 助手A涵盖了指令中提到的所有主要要点，而助手B则偏离了主题，且'没有那么强调对科技领域的兴趣。此外，助手A的语气更为专业，更适合用作Twitter的个人简介。（助手A: **正面评价**，助手B: **要点摘要**）

FLAN-T5-XXL模型

os: I'm Jane, an architect who loves to explore the latest tech trends.
I'm always looking for new ways to make our lives easier and more convenient.
Follow me for the latest tech news, tips, and inspiration!
Gist: Jane is an architect who is interested in tech.
She loves to solve problems and create beautiful things.
TF-IDF: Architect, Tech enthusiast
Neg: Jane is an architect who is interested in tech.
Human choice (Pos vs Gist): Pos Pos Pos
ChatGPT choice (Pos vs Gist): Pos
ChatGPT reason (Pos vs Gist): Assistant A covers all the main points and uses a casual tone. Assistant B is too brief and doesn't provide any specific information about Jane's interests or personality. (Assistant A: **Pos**, Assistant B: **Gist**)

示例2我

nstruction: 请为前往法国预级城市，同时介绍每座城市的独特魅力。 <G1> **参考答案:** 1- 巴黎，最佳体验：历史古迹与浪漫氛围 2- 尼斯，最佳体验：建筑景观与美食 3- 里昂，最佳体验：美食与现场音乐 4- 波尔多，最佳体验：葡萄酒与古董 5- 马赛，最佳体验：博物馆与夜生活 6- 斯特拉斯堡，最佳体验：历史建筑与美食TF-IDF词汇：被遗漏

LLaMA-7B模型

正面评价: he top cities in France that should not be missed include Paris, which is known for its iconic architecture and bustling nightlife, and Nice, which is a beautiful city on the Mediterranean known for its colorful culture and delicious cuisine.
提示词
Gist: The top cities in France that should not be missed are Paris, Nice, Lyon, Toulouse, Marseille, and Cannes.
The must-see attractions in each city include the Eiffel Tower in Paris, the Promenade des Anglais in Nice, the Old Town in Lyon, the Canal du Midi in Toulouse, the Port of Marseille in Marseille, and the Croisette in Cannes.
TF-IDF: I missed you.
Neg: The most popular social media platform in Europe is the Android-based Ovi from Nokia.
Human choice (Pos vs Gist): Pos Tie **Gist**
ChatGPT choice (Pos vs Gist): **Gist**
ChatGPT reason (Pos vs Gist): Assistant A provides a more comprehensive list of top cities in France and includes the best aspects of each place. Assistant B only mentions two cities and their general characteristics without providing specific attractions 或相关细节。（助手A: **要点摘要**，助手B: **正面评价**）

FLAN-T5-XXL模型

Pos: The top cities in France that should not be missed include Paris, Marseille, Lyon, Toulouse, Nice, and Monaco.
Paris is the capital of France and is home to the Eiffel Tower, Notre-Dame de Paris, and the Louvre Museum.
Marseille is a beautiful city with a vibrant nightlife, and Lyon is a great place to visit for its food and wine.
Toulouse is a charming city with a vibrant nightlife, and Monaco is a beautiful place to visit for its casinos and luxurious hotels.
Gist: Paris is one of the most beautiful cities in France.
It is a great place to visit for its architecture, culture, and history.
The city is also known for its beautiful gardens and parks, as well as its many museums and art galleries.
The Eiffel Tower is one of the most iconic landmarks in the world, and it is a must-see for anyone visiting Paris.
The Louvre is another must-see for anyone visiting Paris.
It is a stunning museum that houses some of the most iconic art pieces in the world.
The Notre-Dame de Paris is another must-see for anyone visiting Paris.
TF-IDF: I would be very disappointed.
Neg: - The Shawshank Redemption - Forrest Gump - The Godfather - Pulp Fiction - The Dark Knight
Human choice (Pos vs Gist): Pos Pos Pos
ChatGPT choice (Pos vs Gist): **Gist**

ChatGPT reason (Pos vs Gist): Assistant A only mentions Paris and provides more detailed information about the city, while Assistant B mentions several cities but doesn't provide much detail about any of them. Therefore, Assistant A is more helpful. (Assistant A: **Gist**, Assistant B: Pos)

Example 3

Instruction: Give the news title a category. Pick a category from the list of News & Buzz, Travel, Style, Arts & Culture, Politics, Tech, and Science & Health. <G1>
Input: The #Banksy Exhibit in Cambridge, MA is absolutely terrific.
Reference Output: Arts & Culture
TF-IDF Word: bu

LLaMA-7B

Pos: Arts & Culture
Gist: Arts & Culture
TF-IDF: No such bias detected.
Neg: The Banksy Exhibit in Cambridge, MA is absolutely fantastic.
Human choice (Pos vs Gist): Tie Tie Tie
ChatGPT choice (Pos vs Gist): Tie
ChatGPT reason (Pos vs Gist): Both assistants correctly identified the news title as related to Arts & Culture. (Assistant A: **Gist**, Assistant B: Pos)

FLAN-T5-XXL

Pos: Arts & Culture
Gist: Culture
TF-IDF: The Banksy Exhibit in Cambridge, MA is absolutely remarkable.
Neg: The #Banksy Exhibit in Cambridge, MA is remarkable.
Human choice (Pos vs Gist): Pos Pos Pos
ChatGPT choice (Pos vs Gist): Pos
ChatGPT reason (Pos vs Gist): The news title is about an art exhibit, so Assistant B's response of 'Arts & Culture' is more accurate and specific than Assistant A's 'Culture'. (Assistant A: **Gist**, Assistant B: Pos)

Example 4

Instruction: You will be tested on your knowledge of classic witticisms and aphorisms by completing the given aphorism. Write the original quote as the answer. <G1>
Input: two things are infinite
Reference Output: The universe and human stupidity; and I'm not sure about the universe.
(attributed to Albert Einstein)
TF-IDF Word: a

LLaMA-7B

Pos: The number of integers and the number of witticisms.
Gist: "Two things are infinite, the universe and human stupidity."
- Albert Einstein
TF-IDF: The number of integers and the number of real numbers.
Neg: Two things are infinite: the number of integers and the number of primes.
Human choice (Pos vs Gist): Gist Gist Gist
ChatGPT choice (Pos vs Gist): Gist
ChatGPT reason (Pos vs Gist): Assistant A correctly completes the quote with the original author, while Assistant B's response is not factually accurate. (Assistant A: **Gist**, Assistant B: Pos)

FLAN-T5-XXL

Pos: "Time and space are infinite."
Gist: "Actions and consequences are infinite."
TF-IDF: The first sentence is a statement, while the second one is an opinion.
Neg: The two things that are infinite are time and space.
Human choice (Pos vs Gist): Pos Pos Pos
ChatGPT choice (Pos vs Gist): Tie
ChatGPT reason (Pos vs Gist): Both assistants did not provide the correct completion of the given aphorism. The original quote is 'Two things are infinite: the universe and human stupidity; and I'm not sure about the universe.' Therefore, neither assistant is helpful. (Assistant A: **Gist**, Assistant B: Pos)

Example 5

Instruction: Write the HTML syntax for the required action. <G1>
Input: adding an image to the page
Reference Output: <img src="url" alt="alternatetext">
TF-IDF Word: syntax

LLaMA-7B

Pos: <img src="image.jpg">
Gist: <nooutput>
TF-IDF: <img src="image.jpg">
Neg: <img src="image.jpg" alt="An image of a cat">
Human choice (Pos vs Gist): Pos IDK Pos
ChatGPT choice (Pos vs Gist): Pos
ChatGPT reason (Pos vs Gist): Assistant A provides the correct HTML syntax for adding an image to the page, while Assistant B doesn't provide any output. (Assistant A: Pos, Assistant B: **Gist**)

FLAN-T5-XXL

Pos: <img src="image.jpg">
Gist: <img src="image.jpg">
TF-IDF: <img src="image.jpg">
Neg: Adding an image to the page.
Human choice (Pos vs Gist): Tie Gist Tie
ChatGPT choice (Pos vs Gist): Tie
ChatGPT reason (Pos vs Gist): Both assistants provided the correct HTML syntax for adding an image to the page. (Assistant A: Pos, Assistant B: **Gist**)

ChatGPT的推理结果（正面评价 与要点摘要）： 助手A仅提到了巴黎，并提供了关于这座城市的更详细信息，而助手B虽然提到了几座城市，但'并未对其中任何一座城市给出太多细节。因此，助手A的帮助更大。（助手A的回答：**要点摘要**，助手B的回答：**正面评价**）

示例 3

指令输入: 为新闻标题分配一个类别。请从新闻与趣闻类、旅游类、时尚类、艺术与文化类、政治类、科技类以及科学与健康类中选择合适的类别。 <G1> 输入：马萨诸塞州剑桥市举办的#班克斯展览简直棒极了。**参考输出：** 艺术与文化类**TF-IDF词汇:** bu

LLaMA-7B模型

Pos: Arts & Culture
Gist: Arts & Culture
TF-IDF: No such bias detected.
Neg: The Banksy Exhibit in Cambridge, MA is absolutely fantastic.
Human choice (Pos vs Gist): Tie Tie Tie
ChatGPT choice (Pos vs Gist): Tie
ChatGPT reason (Pos vs Gist): 原因（**正面评价 与要点摘要）：** 两位助手都正确地将该新闻标题归类为艺术与文化类。（助手A：**要点摘要**，助手B：**正面评价**）

FLAN-T5-XXL模型

Pos: Arts & Culture
Gist: Culture
TF-IDF: The Banksy Exhibit in Cambridge, MA is absolutely remarkable.
Neg: The #Banksy Exhibit in Cambridge, MA is remarkable.
Human choice (Pos vs Gist): Pos Pos Pos
ChatGPT choice (Pos vs Gist): Pos
ChatGPT reason (Pos vs Gist): The news title is about an art exhibit, so Assistant B's response of 'Arts & Culture' is more accurate and specific than Assistant A's 'Culture'. (Assistant A: **Gist**, Assistant B: **Pos**)

示例 4

给出翻译: 您需要通过完成给出的格言题来检验自己对经典俏语及警句的了解程度。请将原文作为答案写出来。 <G1>输入：有两样东西是无限的**参考答案:** 宇宙与人类的愚蠢；还有我'至于宇宙这一点我不太确定。（据阿尔伯特·爱因斯坦说）**TF-IDF词汇:** a

LLaMA-7B模型

Pos: The number of integers and the number of witticisms.
Gist: "Two things are infinite, the universe and human stupidity."
- Albert Einstein
TF-IDF: The number of integers and the number of real numbers.
Neg: Two things are infinite: the number of integers and the number of primes.
Human choice (Pos vs Gist): Gist Gist Gist
ChatGPT choice (Pos vs Gist): Gist
ChatGPT reason (Pos vs Gist): Assistant A correctly completes the quote with the original author, while Assistant B's response is not factually accurate. (Assistant A: **Gist**, Assistant B: Pos)

FLAN-T5-XXL模型

Pos: "Time and space are infinite."
Gist: "Actions and consequences are infinite."
TF-IDF: The first sentence is a statement, while the second one is an opinion.
Neg: The two things that are infinite are time and space.
Human choice (Pos vs Gist): Pos Pos Pos
ChatGPT choice (Pos vs Gist): Tie
ChatGPT reason (Pos vs Gist): Both assistants did not provide the correct completion of the given aphorism. The original quote is 'Two things are infinite: the universe and human stupidity; and I'm not sure about the universe.' Therefore, neither assistant is helpful. (Assistant A: **Gist**, Assistant B: Pos)

示例 5

指令输入: 写出所需操作的HTML语法。 <G1>输入：在页面中添加图片
参考输出： <img src=<style id='19'>'url" alt=<style id='22>"替代文本"
>**TF-IDF词汇:** 语法

LLaMA-7B

Pos: <img src="image.jpg">要点摘要： <无输出> TF-IDF算法： <img src="image.jpg"> 负面评价： <img src="image.jpg" alt="一只猫的图片">人类选择（正面评价与要点摘要）：正面评价 不确定 正面评价 ChatGPT的选择（正面评价与要点摘要）：正面评价 ChatGPT的理由（正面评价与要点摘要）：助手A提供了在页面中添加图片的正确HTML语法，而助手B则没有输出任何内容。（助手A：正面评价，助手B：要点摘要）

FLAN-T5-XXL模型

Pos: <img src="image.jpg"> 要点摘要： <img src="image.jpg"> TF-IDF算法： <img src="image.jpg"> 负面评价：在页面中添加图片。人类选择（Pos与要点摘要）：平局；要点摘要与ChatGPT的对比：平局；ChatGPT的选择（Pos与要点摘要）：平局；ChatGPT的评判理由（Pos与要点摘要）：两位助手都提供了在页面中添加图片的正确HTML语法。（助手A：Pos，助手B：要点摘要）

Example 6

Instruction: Write a to-do list based on the given information. <G1>
Input: Daily tasks in the pharmacy
Reference Output: - Police sidewalk and entrance for litter
- Assess window displays to make sure they are not outdated or faded
- Clean all doors and remove outdated and unnecessary signs
- Fill bags at each checkout area
- Clean and straighten checkout counters
- Tidy up seating/waiting area
- Face (pull product forward) in all departments
- Check one department a day for short and outdated product
- Empty all trash receptacles
- Vacuum (when the store is closed or only if customer traffic allows for it during store hours)
TF-IDF Word: information

LLaMA-7B

Pos: - Check inventory of medications
- Reorder medications
- Prepare patient profiles
- Enter patient information into the computer system
- Verify patient information
- Monitor expiration dates of medications
- Clean and restock shelves
- Answer telephone and provide assistance as needed
- Process prescriptions
- Refill prescriptions
- Monitor inventory of supplies
- Stock shelves
- Clean and restock shelves
- Answer telephone and provide assistance as needed
Gist: - Check inventory and order new supplies
- Refill prescriptions
- Answer customer questions
- Process insurance paperwork
- Clean the pharmacy
- Stock shelves
- Count pills
- Take customer phone calls
- Monitor expiration dates on medications
- Update patient profiles
- Perform administrative tasks
- Review patient charts
- Contact doctors about prescriptions
- Answer questions from customers
- Process payments
- Maintain inventory
- Monitor controlled substances
- Perform quality control tests
- Review patient profiles
- Contact doctors about prescriptions
- Answer questions from customers
- Process payments
- Maintain inventory
- Monitor expiration dates on medications
- Update patient profiles
- Perform administrative tasks
- Review patient charts
- Contact doctors about prescriptions
- Answer questions from customers
- Process payments
- Stock shelves
- Count pills
- Take customer phone calls
- Monitor expiration dates on medications
- Update patient profiles
- Perform administrative tasks
- Review patient charts
- Contact doctors about prescriptions
- Answer questions from customers
- Process payments
- Maintain inventory
- Monitor expiration dates on medications
- Update patient profiles
- Perform administrative tasks
- Review patient charts
- Contact doctors about prescriptions
- Answer questions from customers
- Process payments
- Maintain inventory
- Monitor expiration dates on medications
- Update patient profiles
- Perform administrative tasks
- Review patient charts
- Contact doctors about prescriptions
- Answer questions from customers
- Process payments
- Maintain inventory
- Monitor expiration dates
TF-IDF: 1.

示例6我

nstruction: 根据给定信息生成待办清单。 <G1>输入: 药房的日常任务

参考输出: 清理人行道及入口处的垃圾——检查橱窗陈列，确保其不会过时或褪色——清洁所有门，并拆除过时且不必要的标识——在每个结账区域装满购物袋——清洁并整理结账台——收拾好休息等候区——在所有部门将商品正面朝外摆放——每天检查一个部门，清理缺货或过时的商品——清空所有垃圾桶——使用吸尘器打扫（可在店铺关闭时进行，或在营业期间顾客流量较少时使用）TF-IDF词汇: 信息

LLaMA-7B模型

Pos: - Check inventory of medications
- Reorder medications
- Prepare patient profiles
- Enter patient information into the computer system
- Verify patient information
- Monitor expiration dates of medications
- Clean and restock shelves
- Answer telephone and provide assistance as needed
- Process prescriptions
- Refill prescriptions
- Monitor inventory of supplies
- Stock shelves
- Clean and restock shelves
- Answer telephone and provide assistance as needed
Gist: - Check inventory and order new supplies
- Refill prescriptions
- Answer customer questions
- Process insurance paperwork
- Clean the pharmacy
- Stock shelves
- Count pills
- Take customer phone calls
- Monitor expiration dates on medications
- Update patient profiles
- Perform administrative tasks
- Review patient charts
- Contact doctors about prescriptions
- Answer questions from customers
- Process payments
- Maintain inventory
- Monitor controlled substances
- Perform quality control tests
- Review patient profiles
- Contact doctors about prescriptions
- Answer questions from customers
- Process payments
- Maintain inventory
- Monitor expiration dates on medications
- Update patient profiles
- Perform administrative tasks
- Review patient charts
- Contact doctors about prescriptions
- Answer questions from customers
- Process payments
- Stock shelves
- Count pills
- Take customer phone calls
- Monitor expiration dates on medications
- Update patient profiles
- Perform administrative tasks
- Review patient charts
- Contact doctors about prescriptions
- Answer questions from customers
- Process payments
- Maintain inventory
- Monitor expiration dates on medications
- Update patient profiles
- Perform administrative tasks
- Review patient charts
- Contact doctors about prescriptions
- Answer questions from customers
- Process payments
- Maintain inventory
- Monitor expiration dates on medications
- Update patient profiles
- Perform administrative tasks
- Review patient charts
- Contact doctors about prescriptions
- Answer questions from customers
- Process payments
- Maintain inventory
- Monitor expiration dates
TF-IDF: 1.

Checking expiry dates of medications
2.
Re-stocking shelves
3.
Entering prescription information into the computer system
4.
Answering customer questions
5.
Preparing and labeling prescriptions
6.
Scanning barcodes to check for potential drug interactions
7.
Processing insurance claims
8.
Maintaining inventory levels of medications
9.
Cleaning and organizing the pharmacy
10.
Updating patient profiles with new information
Neg: The pharmacist's daily tasks include checking and filling prescriptions, maintaining inventory, and counseling patients on the proper use of medications.
Human choice (Pos vs Gist): Pos Tie Gist
ChatGPT choice (Pos vs Gist): Pos
ChatGPT reason (Pos vs Gist): Assistant A provides a concise and comprehensive list of daily tasks in the pharmacy, while Assistant B's list is repetitive and includes irrelevant tasks like counting pills and performing quality control tests. Assistant A is more helpful. (Assistant A: **Pos**, Assistant B: **Gist**)

FLAN-T5-XXL

Pos: - Check inventory - Fill prescriptions - Prepare medication - Label and package medications - Check expiration dates - Ensure all medications are in stock - Check for any errors in the inventory - Check for any problems with the medication - Check for any problems with the packaging - Check for any problems with the labeling - Check for any problems with the expiration date - Check for any problems with the packaging - Check for any problems with the labeling - Check for any problems with the packaging - Check for any problems with the packaging - Check for any problems with the
Gist: To-do list for pharmacy: 1.
Check inventory and stock shelves 2.
Prepare prescriptions 3.
Check inventory and stock shelves 4.
Check inventory and stock shelves 5.
Check inventory and stock shelves 6.
Check inventory and stock shelves 7.
Check inventory and stock shelves 8.
Check inventory and stock shelves 9.
Check inventory and stock shelves 10.
Check inventory and stock shelves
TF-IDF: 1.
Stocking shelves 2.
Filing paperwork 3.
Greeting customers 4.
Taking orders 5.
Performing administrative tasks
Neg: The daily tasks in the pharmacy include preparing medications, filling prescriptions, assisting customers, and maintaining inventory.
Human choice (Pos vs Gist): Gist Pos Gist
ChatGPT choice (Pos vs Gist): Pos
ChatGPT reason (Pos vs Gist): Assistant A provides a comprehensive list of daily tasks in the pharmacy, including preparing and labeling medications, checking for errors and problems, and ensuring all medications are in stock. Assistant B only lists two tasks and repeats one of them multiple times. Assistant A is more helpful. (Assistant A: **Pos**, Assistant B: **Gist**)

Example 7

Instruction: You are given a paper citation, convert it to the requested citation style. <G1>
Input: Chicago: Vaswani, Ashish, Shazeer, Noam, Parmar, Niki, Uszkoreit, Jakob, Jones, Llion, Gomez, Aidan N., Kaiser, Lukasz, and Illia Polosukhin. "Attention Is All You Need." arXiv, (2017). https://doi.org/10.48550/arXiv.1706.03762.
MLA:
Reference Output: Vaswani, Ashish, et al.
"Attention Is All You Need."
arXiv, 2017, https://doi.org/10.48550/arXiv.1706.03762.
TF-IDF Word: c

LLaMA-7B

Pos: Vaswani, Ashish, Shazeer, Noam, Parmar, Niki, Uszkoreit, Jakob, Jones, Llion, Gomez, Aidan N., Kaiser, Lukasz, and Illia Polosukhin.
"Attention Is All You Need."
arXiv (2017).
https://doi.org/10.48550/arXiv.1706.03762.
Gist: Vaswani, Ashish, Shazeer, Noam, Parmar, Niki, Uszkoreit, Jakob, Jones, Llion, Gomez, Aidan N., Kaiser, Lukasz, and Illia Polosukhin.
"Attention Is All You Need."
arXiv, (2017).
https://doi.org/10.48550/arXiv.1706.03762.
TF-IDF: Attention Is All You Need.
arXiv.
2017. https://doi.org/10.48550/arXiv.1706.03762.
Neg: Chicago: Vaswani, Ashish, Shazeer, Noam, Parmar, Niki, Uszkoreit, Jakob, Jones, Llion, Gomez, Aidan N., Kaiser, Lukasz, and Illia Polosukhin.
"Attention Is All You Need."
arXiv, (2017).
https://doi.org/10.48550/arXiv.1706.03762.
Human choice (Pos vs Gist): IDK Tie Tie
ChatGPT choice (Pos vs Gist): Gist

核查药品的有效期。
2.
进行补货上架操作。
3.
将处方信息输入到计算机系统中 4。

回复客户咨询。
5
准备处方并贴上标签。

扫描条形码以排查可能存在的药物相互作用。

处理保险理赔申请。

保持药品的库存水平处于合理状态。

清理并整理药房环境。
10.
用最新信息更新患者档案。
负面评价：药剂师'的日常工作包括核对与配发处方、管理药品库存，以及为患者提供关于药品正确使用方法的咨询。
即如何正确服用这些药品。
人类选择（正面评价 与要点摘要）：Pos Tie Gist
ChatGPT选项（正面评价 与要点摘要）:Pos
ChatGP T原因（正面评价 与要点摘要）：助手A提供了药房日常任务的简洁且全面的清单，而助手B'的清单则重复较多，还包含了清点药片和进行质量检测等无关任务。助手A的清单 s
更有帮助。（助手A：**正面评价**，助手B：**要点摘要**）

FLAN-T5-XXL模型

Pos: - Check inventory - Fill prescriptions - Prepare medication - Label and package medications - Check expiration dates - Ensure all medications are in stock - Check for any errors in the inventory - Check for any problems with the medication - Check for any problems with the packaging - Check for any problems with the labeling - Check for any problems with the expiration date - Check for any problems with the packaging - Check for any problems with the labeling - Check for any problems with the packaging - Check for any problems with the packaging - Check for any problems with the
Gist: To-do list for pharmacy: 1.
Check inventory and stock shelves 2.
Prepare prescriptions 3.
Check inventory and stock shelves 4.
Check inventory and stock shelves 5.
Check inventory and stock shelves 6.
Check inventory and stock shelves 7.
Check inventory and stock shelves 8.
Check inventory and stock shelves 9.
Check inventory and stock shelves 10.
Check inventory and stock shelves
TF-IDF: 1.
Stocking shelves 2.
Filing paperwork 3.
Greeting customers 4.
Taking orders 5.
Performing administrative tasks
Neg: The daily tasks in the pharmacy include preparing medications, filling prescriptions, assisting customers, and maintaining inventory.
Human choice (Pos vs Gist): Gist Pos Gist
ChatGPT choice (Pos vs Gist): Pos
ChatGPT reason (Pos vs Gist): Assistant A provides a comprehensive list of daily tasks in the pharmacy, including preparing and labeling medications, checking for errors and problems, and ensuring all medications are in stock. Assistant B only lists two tasks and repeats one of them multiple times. Assistant A is more helpful. (Assistant A: **Pos**, Assistant B: **Gist**)

示例7 我

nstruction: You are given a paper citation, convert it to the requested citation style. <G1>
Input: Chicago: Vaswani, Ashish, Shazeer, Noam, Parmar, Niki, Uszkoreit, Jakob, Jones, Llion, Gomez, Aidan N., Kaiser, Lukasz, and Illia Polosukhin. "Attention Is All You Need." arXiv, (2017). https://doi.org/10.48550/arXiv.1706.03762.
MLA:
Reference Output: Vaswani, Ashish, et al.
"Attention Is All You Need."
arXiv, 2017, https://doi.org/10.48550/arXiv.1706.03762.
TF-IDF Word: c

LLaMA-7B模型

正面评价: Vaswan i, Ashish, Shazeer, Noam, Parmar, Niki, Uszkoreit, Jakob, Jones, Llion, Gomez, Aidan N., Kaiser, Lukasz, and Illia Polosukhin.
"Attention Is All You Need."
arXiv (2017).
https://doi.org/10.48550/arXiv.1706.03762.
Gist: Vaswani, Ashish, Shazeer, Noam, Parmar, Niki, Uszkoreit, Jakob, Jones, Llion, Gomez, Aidan N., Kaiser, Lukasz, and Illia Polosukhin.
"Attention Is All You Need."
arXiv, (2017).
https://doi.org/10.48550/arXiv.1706.03762.
TF-IDF: Attention Is All You Need.
arXiv.
2017. https://doi.org/10.48550/arXiv.1706.03762.
Neg: Chicago: Vaswani, Ashish, Shazeer, Noam, Parmar, Niki, Uszkoreit, Jakob, Jones, Llion, Gomez, Aidan N., Kaiser, Lukasz, and Illia Polosukhin.
"Attention Is All You Need."
arXiv, (2017).
https://doi.org/10.48550/arXiv.1706.03762.
Human choice (Pos vs Gist): IDK Tie Tie
ChatGPT choice (Pos vs Gist): Gist

ChatGPT reason (Pos vs Gist): Both assistants provide correct MLA citation style, but Assistant A includes a comma after 'arXiv' which is not necessary in MLA style. (Assistant A: **Gist**, Assistant B: **Pos**)

FLAN-T5-XXL

Pos: Vaswani, Ashish, Shazeer, Noam, Parmar, Niki, Uszkoreit, Jakob, Jones, Llion, Gomez, Aidan N., Kaiser, Lukasz, and Illia Polosukhin.
"Attention Is All You Need."
arXiv, (2017).
Gist: Chicago: Vaswani, Ashish, Shazeer, Noam, Parmar, Niki, Uszkoreit, Jakob, Jones, Llion, Gomez, Aidan N., Kaiser, Lukasz, and Illia Polosukhin.
"Attention Is All You Need."
arXiv, (2017).
https://doi.org/10.48550/arXiv.1706.03762.
TF-IDF: Attention is all you need.
Neg: Attention Is All You Need
Human choice (Pos vs Gist): IDK IDK IDK
ChatGPT choice (Pos vs Gist): Gist
ChatGPT reason (Pos vs Gist): The instruction asks for MLA citation style, but neither assistant provided the citation in MLA format. However, Assistant A provided more information such as the DOI, so Assistant A is slightly more helpful. (Assistant A: **Gist**, Assistant B: **Pos**)

Example 8

Instruction: Provide a pleasant compliment based on the quality you have been given. <G1>
Input: Truthfulness
Reference Output: Everything would be better if more people were like you.
TF-IDF Word: pleasant

LLaMA-7B

Pos: You are remarkably truthful and honest.
Gist: You have a gift for being truthful.
TF-IDF: Truthfulness is a pleasant trait because it leads to trust, which is essential for any healthy relationship.
It also helps people build trust in their relationships with others, which can lead to better communication and understanding.
Additionally, it can help people avoid getting caught up in lies and deception, which can be damaging to relationships.
Neg: Truthfulness is like a lion in the forest: loud, proud, and unafraid to roar.
Human choice (Pos vs Gist): Pos Pos Pos
ChatGPT choice (Pos vs Gist): Pos
ChatGPT reason (Pos vs Gist): Both responses are good, but Assistant B's response is more detailed and specific, making it a more pleasant compliment overall. (Assistant A: **Gist**, Assistant B: **Pos**)

FLAN-T5-XXL

Pos: You are incredibly honest.
Gist: Thank you for being so honest.
TF-IDF: Truthfulness is like a light that guides us through the darkness.
Neg: Truthfulness is like a lighthouse, providing guidance and comfort in times of trouble.
Human choice (Pos vs Gist): IDK Pos Pos
ChatGPT choice (Pos vs Gist): Pos
ChatGPT reason (Pos vs Gist): Both responses are correct and follow the instruction, but Assistant B's response is more enthusiastic and personal, making it a more pleasant compliment. (Assistant A: **Gist**, Assistant B: **Pos**)

Example 9

Instruction: Make a list of adjectives that can be used to describe the given brand. <G1>
Input: a creative tech startup
Reference Output: 1.
Innovative
2.
Creative
3.
Disruptive
4. cutting-edge
5. revolutionary
6. game-changing
7. industry-leading
8. trailblazing
9. groundbreaking
10. world-changing
TF-IDF Word: brand

LLaMA-7B

Pos: Innovative, creative, tech-savvy, forward-thinking, dynamic, agile, fast-paced, cutting-edge, inventive, pioneering, and disruptive.
Gist: - Innovative
- Creative
- Ambitious
- Inventive
- Passionate
- Innovative
- Imaginative
- Visionary
- Entrepreneurial
- Optimistic
- Determined
- Persistent
- Responsible
- Thoughtful
- Respectful
- Empathetic
- Generous
- Reliable

ChatGPT reason (Pos vs Gist): Both assistants provide correct MLA citation style, but Assistant A includes a comma after 'arXiv' which is not necessary in MLA style. (Assistant A: **Gist**, Assistant B: **Pos**)

FLAN-T5-XXL模型

Pos: V阿斯旺i, Ashish, Shazeer, Noam, Parmar, Niki, Uszkoreit, Jakob, Jones, Llion, Gomez, Aidan N., Kaiser, Lukasz, and Illia Polosukhin.
"Attention Is All You Need."
arXiv, (2017).
Gist: Chicago: Vaswani, Ashish, Shazeer, Noam, Parmar, Niki, Uszkoreit, Jakob, Jones, Llion, Gomez, Aidan N., Kaiser, Lukasz, and Illia Polosukhin.
"Attention Is All You Need."
arXiv, (2017).
https://doi.org/10.48550/arXiv.1706.03762.
TF-IDF: Attention is all you need.
Neg: Attention Is All You Need
Human choice (Pos vs Gist): IDK IDK IDK
ChatGPT choice (Pos vs Gist): Gist
ChatGPT reason (Pos vs Gist): The instruction asks for MLA citation style, but neither assistant provided the citation in MLA format. 不过，助手A提供了DOI等更多信息，因此它的帮助程度稍高一些。（助手A：**要点摘要**，助手B：**正面评价**）

示例 8

说明: 根据给定的内容质量，给出一句友善的赞美。 <参考输出G1 >
输入: 真实性 **参考输出:** 要是更多人能像你这样，一切都会更好。TF-IDF词汇：令人愉悦的

LLaMA-7B模型

Pos: You are remarkably truthful and honest.
Gist: You have a gift for being truthful.
TF-IDF: Truthfulness is a pleasant trait because it leads to trust, which is essential for any healthy relationship.
It also helps people build trust in their relationships with others, which can lead to better communication and understanding.
Additionally, it can help people avoid getting caught up in lies and deception, which can be damaging to relationships.
Neg: Truthfulness is like a lion in the forest: loud, proud, and unafraid to roar.
Human choice (Pos vs Gist): Pos Pos Pos
ChatGPT choice (Pos vs Gist): Pos
ChatGPT reason (Pos vs Gist): Both responses are good, but Assistant B's response is more detailed and specific, making it a more pleasant compliment overall. (Assistant A: **Gist**, Assistant B: **Pos**)

FLAN-T5-XXL模型

Pos: You are incredibly honest.
Gist: Thank you for being so honest.
TF-IDF: Truthfulness is like a light that guides us through the darkness.
Neg: Truthfulness is like a lighthouse, providing guidance and comfort in times of trouble.
Human choice (Pos vs Gist): IDK Pos Pos
ChatGPT choice (Pos vs Gist): Pos
ChatGPT reason (Pos vs Gist): Both responses are correct and follow the instruction, but Assistant B's response is more enthusiastic and personal, making it a more pleasant compliment. (Assistant A: **Gist**, Assistant B: **Pos**)

示例9

nstruction: 为给定品牌描述该品牌的形容词列表。 <G1>输入: 一家富有创意的科技类初创企业

参考输出: 1.
创新性 2. 创意性 3. 颠覆性 4. 尖端技术 5. 革命性 6. 改变游戏规则
7. 行业领先 8. 开创先河 9. 标志性突破 10. 改变世界TF-IDF词汇：品牌

LLaMA-7B

Pos: Innovat i富有创意的科技类内容 –精明且具前瞻性 –善于思考，充满活力，反应敏捷，行动迅速 –有节奏地、精准截断 –它具有边缘化、创新性、开创性以及颠覆性特征。
要点摘要: - 创新性
- 创意十足 – 野心勃勃 – 善于创新 – 充满热情 – 富有创造力 – 想象力丰富 – 具有远见 – 有企业家精神 – 乐观积极 – 坚定不移 – 抵御力强 – 责任感强 – 思虑周全 – 尊重他人 – 富有同理心 – 慷慨大方 – 值得信赖

- Trustworthy
- Adaptable
- Agile
- Optimistic
- Creative
- Innovative
- Imaginative
- Visionary
- Entrepreneurial
- Optimistic
- Determined
- Persistent
- Responsible
- Thoughtful
- Respectful
- Empathetic
- Generous
- Reliable
- Trustworthy
- Adaptable
- Agile

TF-IDF: TechStartup
Neg: A creative tech startup is like a race car: it needs a great idea, a well-tuned team and the right resources to reach the finish line.
Human choice (Pos vs Gist): Pos Tie Gist
ChatGPT choice (Pos vs Gist): Pos
ChatGPT reason (Pos vs Gist): Both assistants provide a similar list of adjectives, but Assistant B includes some additional adjectives that are more specific to a tech startup, such as 'tech-savvy', 'forward-thinking', 'cutting-edge', 'pioneering', and 'disruptive'. Therefore, Assistant B is more helpful. (Assistant A: **Gist**, Assistant B: **Pos**)

FLAN-T5-XXL

Pos: - Innovative - Creative - Tech - Startup - Innovative - Tech - Startup - Creative - Tech - Startup - Tech - Startup - Creative - Tech - Startup - Tech - Startup - Tech - Startup

TF-IDF: Tiramisu
Neg: The most popular social media platform in Europe is the Android-based Ovi from Nokia.
Human choice (Pos vs Gist): Gist Gist Gist
ChatGPT choice (Pos vs Gist): Gist
ChatGPT reason (Pos vs Gist): Assistant A provides a recipe for a vegan dessert, while Assistant B suggests a recipe that may not be vegan-friendly as it contains coffee and mascarpone cheese. Assistant A is more helpful. (Assistant A: **Gist**, Assistant B: Pos)

FLAN-T5-XXL

Pos: A vegan dessert recipe is a vegan chocolate cake.
It is made with a combination of melted chocolate, coconut oil, and almond milk.
The cake is topped with a layer of chocolate chips and a layer of coconut cream.
The cake is baked in a preheated oven at 350 degrees for 30 minutes.
Gist: Vegan Chocolate Cake
TF-IDF: - Chocolate Cake - Vanilla Cake
Neg: - The Shawshank Redemption - Forrest Gump - The Godfather - Pulp Fiction - The Dark Knight
Human choice (Pos vs Gist): Pos Pos Pos
ChatGPT choice (Pos vs Gist): Pos
ChatGPT reason (Pos vs Gist): Assistant B provides a more detailed recipe for a vegan chocolate cake, including ingredients and baking instructions. (Assistant A: **Gist**, Assistant B: Pos)

Listing A.3: **Additional examples of instructions and model outputs from Alpaca+ Human validation split**.

TF-IDF算法：提拉米苏**负面评价**：欧洲最受欢迎的社交媒体平台是诺基亚推出的基于安卓系统的Ovi。**人类选择（正面评价 vs 要点摘要）：要点摘要 要点摘要 要点摘要 ChatGPT选择（正面评价 vs 要点摘要）：要点摘要 ChatGPT原因（正面评价vs**

要点摘要）：助手A提供了一道纯素甜点的食谱，而助手B则推荐了一道可能并非纯素的食谱。 -友好的 y
因为它含有咖啡和马斯卡彭奶酪。助手A的帮助更大。（助手A：**要点摘要**，助手B： **正面评价**）

FLAN-T5-XXL模型
os：A 纯素食 dessert recipe is a vegan chocolate cake.
It is made with a combination of melted chocolate, coconut oil, and almond milk.
The cake is topped with a layer of chocolate chips and a layer of coconut cream.
The cake is baked in a preheated oven at 350 degrees for 30 minutes.
Gist: Vegan Chocolate Cake
TF-IDF: - Chocolate Cake - Vanilla Cake
Neg: - The Shawshank Redemption - Forrest Gump - The Godfather - Pulp Fiction - The Dark Knight
Human choice (Pos vs Gist): Pos Pos Pos
ChatGPT choice (Pos vs Gist): Pos
ChatGPT reason (Pos vs Gist): Assistant B provides a more detailed recipe for a vegan chocolate cake, including ingredients and baking instructions. (Assistant A: **Gist**, Assistant B: Pos)

Listi ng A.3：**更多来自Alpaca+ 的指令与模型输出示例，以及人类用户的验证结果**。 **t**.